

LAYLA: ESCRITURA CREATIVA

DOCUMENTO DE PROYECTO

LUIS DONALDO ORTIZ GARCÍA
RODOLFO FERNANDEZ RODRÍGUEZ

DESARROLLO DE SISTEMAS EN RED Y DESPLIEGUE DE SOFTWARE
LICENCIATURA EN INGENIERÍA DE SOFTWARE
PERIODO FEBRERO – JUNIO 2026

Contenido

Anexos.....	3
Historial de versiones	4
Introducción.....	6
Planificación y Cronograma de Desarrollo.....	6
Glosario de Términos y Definiciones	8
Requisitos.....	10
Contexto	10
Clases de usuario.....	10
Casos de uso (análisis)	12
CU-01: Explorar Catalogo Publico	12
CU-02: Previsualizar Sinopsis	12
CU-03: Iniciar Sesión / Registrarse	13
CU-04: Gestionar Perfil	13
CU-05: Crear Proyecto Narrativo	13
CU-06: Gestionar Colaboradores	14
CU-07: Configurar Privacidad	14
CU-08: Editar Manuscrito	15
CU-09: Gestionar Wiki	15
CU-10: Visualizar Grafo Narrativo	15
CU-11: Participar en Voz como Hablante	16
CU-12: Unirse a Voz como Oyente	16
CU-13: Leer Historia Publicada	16
CU-14: Consultar Reportes del Sistema	17
CU-15: Gestionar Usuarios.....	17
Prototipo de UI	18
Cliente de Escritorio	19
Estrategia de Implementación de UI (Enfoque MVP Académico)	24
Requisitos funcionales.....	26
Requisitos no funcionales	28
Diseño.....	30
Diseño arquitectónico	30

Vista de casos de uso (Diagrama UML de casos de uso).....	30
Vista lógica (Diagrama de clases).....	31
Vista de implementación (Diagrama de componentes)	31
Vista de procesos.....	33
Diagramas de Secuencia	33
Diagramas de Estado	35
Vista de despliegue (Diagrama de despliegue)	40
Control de Acceso y Seguridad (Matriz de roles)	40
Estrategia de Clientes y Especialización de Interfaces	41
Abordaje de Retos de Sistemas Distribuidos.....	42
Uso de Arquitectura Orientada a Eventos	43
Análisis y Gestión de Riesgos Técnicos	44
Modelo de datos (Diagrama de entidad-relación)	47
Descripciones de casos de uso	47
Construcción	53
Selección justificada de la pila tecnológica	53
Aplicación de cliente rico	54
Aplicación de servicios (API) - Servidor	55
Estándares de codificación.....	57
C#	57
JavaScript.....	57
Kotlin	58
Reportes de análisis estático de código.....	58
Prácticas de construcción realizadas.....	58
Pruebas	59
Plan de pruebas para la Aplicación de servicios (API) y clientes.....	59
Procedimiento de prueba (automatizado).....	60
Casos de prueba (automatizados)	61
Resultados	65
Estrategia de despliegue	66
Conclusiones	67
Apéndices.....	68

Apéndice A: Roadmap Tecnológico y Evolución a Producto Comercial (Layla 2.0)	68
--	----

Anexos

Diagrama 1 - Diagrama de contexto del sistema.	10
Diagrama 2 - Flujos de vistas implementados.	18
Diagrama 3 - Modelado de casos de uso y actores.	30
Diagrama 4 - Representación de clases de despliegue virtual.	31
Diagrama 5 - Representación de componentes en implementación.	31
Diagrama 6 - Secuencia: Inicio de sesión con token JWT.	33
Diagrama 7 - Secuencia: Crear proyecto.	33
Diagrama 8 - Secuencia: Edición colaborativa de manuscrito en tiempo real.	34
Diagrama 9 - Secuencia: Sala de voz del proyecto.	35
Diagrama 10 - Estado: Conexión a colaboración.	35
Diagrama 11 - Estado: Autenticación del usuario.	36
Diagrama 12 - Estado: Edición colaborativa.	37
Diagrama 13 - Estado: Estado en sala de voz.	38
Diagrama 14 - Estado: Cambio de visibilidad del proyecto.	39
Diagrama 15 - Despliegue: Exposición de los protocolos y puertos.	40
Prototipo 1 - Inicio de sesión.	19
Prototipo 2 - Registro de usuario nuevo.	19
Prototipo 3 - Consulta de "Mis proyectos".	20
Prototipo 4 - Consulta de proyectos públicos.	20
Prototipo 5 - Creación de un nuevo proyecto.	21
Prototipo 6 - Configuración del perfil propio del usuario.	21
Prototipo 7 - Espacio de trabajo para la creación de manuscritos.	22
Prototipo 8 - Espacio de trabajo para la creación de wiki.	22
Prototipo 9 - Espacio de trabajo para manejo de grafos.	23
Prototipo 10 - Espacio de trabajo de la sala de voz.	23
Prototipo 11 - Espacio de trabajo del proyecto (solo permiso de lectura).	24

Historial de versiones

Versión	Fecha	Resumen
1.0	11 de febrero de 2026	• Creación de estructura inicial e introducción • Creación de matriz de acceso • Creación de clases de usuario • Creación de diagrama de contexto • Creación de tabla de clases de usuario y dispositivos • Creación de RF • Creación de RNF • Creación de tabla de selección de pila tecnológica • Creación de tabla de clases de usuario y dispositivos • Creación de plan de pruebas (inicial y superficial) • Creación de lista de casos de uso • Creación de diagrama de casos de uso
1.1	12 de febrero de 2026	• Actualización de sección Introducción • Actualización de tabla de clases de usuario • Actualización de RF • Creación de sección “Node.js y gRPC” • Actualización de tabla de selección de pila tecnológica • Nota sobre estrategia de despliegue agregada. • Actualización de RF • Actualización de RNF • Creación de sección “Uso de Arquitectura Orientada a Eventos • Eliminación de sección “Node.js y gRPC”) • Vista de despliegue agregada
1.2	16 de febrero de 2026	• Creación de casos de uso del módulo de gestión y seguridad (CU-05, CU-06, CU-07). • Creación de descripción para producción literaria y edición de manuscritos (CU-08). • Creación de casos de uso de Worldbuilding y gestión de entidades (CU-09, CU-10).

		• Creación de casos de uso del módulo de voz y colaboración (CU-11, CU-12). • Creación de casos de uso del módulo de lectura pública (CU-01, CU-02, CU-13).
1.3	17 de febrero de 2026	• Definición de la estrategia de implementación de UI (MVP Académico). • Análisis del flujo de navegación y propuesta de simplificación de componentes. • Creación de la matriz de riesgos técnicos y estrategias de mitigación (R-01 a R-06). • Definición de la estrategia de despliegue y arquitectura de contenedores. • Configuración de mecanismos de resiliencia y autorrecuperación (Self-Healing). • Creación del Apéndice A: Roadmap Tecnológico y Evolución a Layla 2.0.

Introducción

El presente proyecto, titulado "**Layla: Ecosistema de Escritura Creativa y Colaboración**", surge como una solución integral para autores y constructores de universos (*worldbuilders*) que requieren un entorno centralizado para la gestión narrativa.

Desde una perspectiva técnica, Layla se desarrolla como un **sistema distribuido heterogéneo** que implementa una arquitectura de microservicios para garantizar la escalabilidad y la tolerancia a fallos. La plataforma se compone de un ecosistema híbrido:

1. **Núcleo Transaccional (.NET 10):** Gestiona la identidad, seguridad y orquestación de sesiones de voz.
2. **Microservicio de Worldbuilding (Node.js):** Un servicio especializado encargado de la gestión de grafos de conocimiento (Neo4j) y documentos extensos (MongoDB).
3. **Comunicación Inter-Servicios:** Ambos backends se comunican mediante **RabbitMQ**.

Para la interfaz de usuario, se ha optado por una estrategia de **Tres Pantallas**:

1. **Cliente de Escritorio (WPF):** Una aplicación nativa de alto rendimiento para la redacción intensiva y gestión de manuscritos sin distracciones.
2. **Cliente Web (Blazor Server):** Un portal administrativo y de lectura pública para la gestión de usuarios y difusión de contenido.
3. **Aplicación Móvil (Android):** cliente reducido para autenticación, administración ligera de proyectos propios y consulta de métricas del sistema.

Planificación y Cronograma de Desarrollo

Fase	Semana (Est.)	Hito (Milestone)	Entregable Técnico
I. Infraestructura	Semana 1	Despliegue de Contenedores	docker-compose operativo con SQL Server, Mongo y API .NET respondiendo HTTP 200 OK.

	Semana 2	Identidad y Seguridad	Sistema de Login/Registro funcional con emisión de JWT. Cliente WPF con pantalla de autenticación.
II. Núcleo (Escritura)	Semana 3	Gestión de Proyectos	CRUD de Proyectos en WPF conectado a SQL Server. Persistencia de metadatos básicos.
	Semana 4	Editor de Texto	Integración de RichTextBox en WPF. Guardado de contenido extenso en MongoDB.
	Semana 5	Modo Offline	Implementación de caché local en WPF y sincronización básica al recuperar conexión.
III. Distribuido	Semana 6	Servicio de Worldbuilding	Despliegue del contenedor Node.js y conexión exitosa con Neo4j (Creación de nodos).
	Semana 7	Sincronización Asíncrona	Implementación de RabbitMQ. Evento ProjectCreated en .NET dispara creación de nodo en Neo4j.
IV. Clientes Satélite	Semana 8	Portal Web de Lectura	Cliente Blazor Server desplegado. Visualización de lista de proyectos públicos (Feed).
	Semana 9	Cliente Movil (Android)	App nativa Kotlin conectada a la API. Login, gestión de proyectos propios y consulta de métricas del sistema.
V. Tiempo Real	Semana 10	Servidor de Chat	Implementación de Hub de WebSockets en .NET.

			Pruebas de conexión de sockets concurrentes.
	Semana 11	Voz en clientes principales	Módulo de voz Push-to-Talk en escritorio y web mediante SignalR/WebSockets.
VI. Cierre	Semana 12	Resiliencia y Defensa	Configuración de `restart` policies en Docker. Pruebas de estrés y documentación final.

Glosario de Términos y Definiciones

Términos del Dominio Narrativo (Negocio)

Término	Definición
Escaleta (Beat Sheet)	Estructura esquemática de la narración que desglosa la historia en eventos clave o "golpes" (beats) antes de escribir el texto completo. En Layla, funciona como el índice dinámico del proyecto.
Manuscrito	El cuerpo textual principal de la obra (capítulos y escenas). Es el contenido no estructurado que se almacena en documentos de texto enriquecido (Rich Text).
Worldbuilding	Proceso de construcción de un mundo imaginario (geografía, historia, cultura). En el sistema, se refiere al módulo que gestiona la Wiki y el Grafo de relaciones.
Wiki (Códice)	Base de conocimientos del proyecto donde se almacenan fichas de personajes, lugares y objetos. Está vinculada directamente al grafo de relaciones.
Grafo Narrativo	Representación visual y lógica de las conexiones semánticas entre las entidades del mundo (ej. <i>"Personaje A odia a Personaje B"</i>). Se implementa técnicamente en Neo4j.
Proyecto	La unidad contenedora raíz que agrupa manuscritos, escaletas, wikis y configuraciones de permisos. Equivale a una "Novela" o "Saga".

Términos Técnicos y de Arquitectura

Término	Definición
Sincronización Híbrida	Capacidad del Cliente de Escritorio (WPF) para almacenar cambios localmente (Offline First) y transmitirlos al servidor automáticamente cuando se restablece la conexión a internet.
Consistencia Eventual	Modelo de consistencia de datos utilizado en la arquitectura distribuida. Garantiza que, tras una actualización en el Backend Core (.NET), los cambios se propagarán al servicio de Worldbuilding (Node.js) mediante colas de mensajes (RabbitMQ), aunque exista un leve retraso temporal.
Push-to-Talk (PTT)	Mecanismo de transmisión de voz donde el usuario debe mantener presionado un botón para enviar audio. Se utiliza para evitar el eco y reducir el consumo de ancho de banda en la aplicación móvil.
Cliente Rico (Rich Client)	Referencia a la aplicación de escritorio (WPF) que procesa gran parte de la lógica de presentación y renderizado en el hardware del usuario, a diferencia de un cliente ligero (Web).
Cliente móvil reducido	Rol actual de Android: autenticación, administración ligera de proyectos propios y consulta de métricas. No cubre manuscritos, wiki, grafo ni voz.
Nodo (Infraestructura)	En el contexto de despliegue, se refiere a una instancia de contenedor o servidor (ej. "Nodo de RabbitMQ"). Nota: No confundir con "Nodo del Grafo Narrativo".

Actores y Roles

Término	Definición
Owner (Propietario)	El creador del proyecto. Tiene permisos destructivos (borrar obra) y de gestión total de accesos.
Oyente (Listener)	Estado de un usuario en el módulo de voz que puede recibir audio pero no tiene permisos para transmitir (micrófono deshabilitado por el sistema).

Feed Público	Catálogo de obras marcadas con visibilidad "Pública" accesible para usuarios Invitados y Lectores, servido desde una caché de lectura optimizada.
---------------------	---

Requisitos

Contexto

Layla conecta a autores, lectores y administradores mediante tres clientes especializados que acceden por el API Gateway. El gateway enruta las operaciones de identidad, proyectos y tiempo real hacia server-core, y las operaciones de manuscritos, wiki y grafo hacia server-worldbuilding. SQL Server, MongoDB, Neo4j y RabbitMQ son sistemas externos al límite funcional del sistema Layla y sostienen persistencia, grafo narrativo y eventos asíncronos.

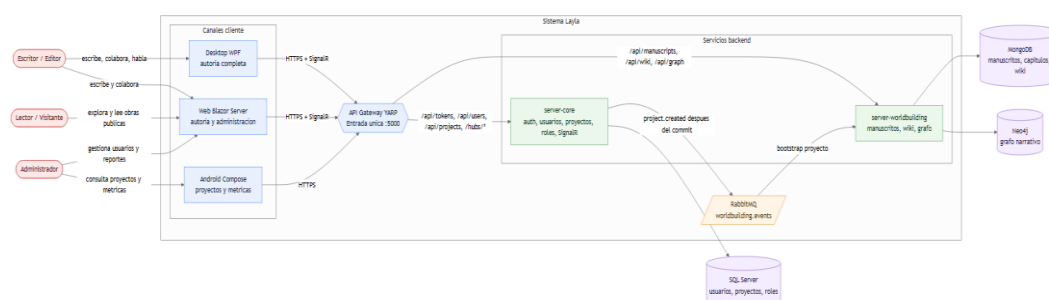


Diagrama 1 - Diagrama de contexto del sistema.

Clases de usuario

1. Escritor (Writer / Owner)

Es el usuario con el nivel de autoridad más alto sobre un proyecto específico.

- **Funciones principales:** Crear y eliminar proyectos, gestionar la estructura jerárquica de la obra (escaletas), redactar manuscritos con texto enriquecido y administrar los permisos de otros colaboradores.
- **Capacidad de comunicación:** Tiene acceso total al módulo de voz vía WebSockets para transmitir audio (hablar) y recibir audio (escuchar) en sesiones de coescritora.
- **Persistencia:** Gestiona directamente registros en SQL Server (metadatos del proyecto), MongoDB (contenido) y Neo4j (grafos de personajes).

2. Editor (Collaborator)

Es un usuario invitado por el Escritor para trabajar activamente en una obra ajena.

- **Funciones principales:** Modificar el contenido de los manuscritos y actualizar las fichas de la Wiki o las relaciones en el grafo de personajes. No puede eliminar el proyecto ni cambiar los roles de otros usuarios.
- **Capacidad de comunicación:** Posee permisos de audio bidireccional (hablar y escuchar) para participar en las lluvias de ideas en tiempo real.
- **Seguridad:** Su acceso está limitado mediante el token JWT a operaciones de tipo UPDATE y POST sobre el contenido narrativo.

3. Lector Registrado (Registered Reader)

Es un usuario autenticado que consume el contenido de la plataforma.

- **Funciones principales:** Consultar manuscritos terminados, navegar por las Wikis y visualizar los mapas de relaciones de personajes.
- **Capacidad de comunicación:** Puede unirse a las sesiones de voz como oyente (solo escucha), permitiéndole seguir transmisiones de autores, pero sin capacidad de transmitir audio.
- **Restricciones:** El sistema bloquea cualquier intento de escritura o modificación de datos.

4. Invitado (Guest / Anonymous)

Usuario no autenticado que accede al sistema (similar a la navegación inicial en Pinterest).

- **Funciones principales:** Visualizar exclusivamente el contenido que los Escritores han marcado como "Público" (sinopsis, portadas o muestras gratis de capítulos).
- **Restricciones:** No posee un token JWT, por lo que el Backend rechaza automáticamente cualquier petición a los módulos de voz, edición o wikis privadas. Su objetivo es incentivar el registro en la plataforma.

Tabla de clases de usuario y dispositivos

Clase de Usuario	Dispositivo Principal	Canal Técnico	Actividad Clave
Escritor	Escritorio WPF / Web Blazor	HTTPS + SignalR	Producción: Redacción de manuscritos, wiki, grafo, voz y gestión de colaboradores.
Editor	Escritorio WPF / Web Blazor	HTTPS + SignalR	Coautoría: Edición de capítulos, wiki y relaciones narrativas según permisos.

Administrador	Web Blazor / Android reducido	HTTPS	Supervisión: Gestión de usuarios, proyectos y métricas del sistema.
Lector	Web Blazor / Escritorio WPF	HTTPS	Consumo: Exploración de obras públicas y lectura de contenido autorizado.
Usuario móvil	Android Compose	HTTPS	Administración ligera: Login, proyectos propios y consulta de métricas cuando tiene rol Admin.

Casos de uso (análisis)

Las siguientes descripciones pertenecen al nivel de análisis: expresan objetivos, actores y comportamiento observable sin depender de clases, endpoints, bases de datos ni decisiones de implementación. La sección de diseño reutiliza la misma estructura y agrega los detalles técnicos.

CU-01: Explorar Catalogo Publico

Actor principal	Visitante / Lector
Objetivo	Descubrir obras marcadas como públicas.
Precondiciones	El sistema está disponible y existen proyectos públicos.
Flujo básico	El usuario entra al catálogo, revisa obras disponibles, filtra o busca por interés y selecciona una obra.
Flujos alternativos	Si no hay obras públicas, el sistema muestra un estado vacío informativo.
Postcondiciones	El usuario identifica una obra para previsualizar o leer.
Casos relacionados	Puede conducir a CU-02 o CU-03.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-02: Previsualizar Sinopsis

Actor principal	Visitante / Lector
Objetivo	Conocer la información pública de una obra antes de entrar al contenido completo.
Precondiciones	El usuario selecciono una obra pública del catálogo.
Flujo básico	El sistema muestra portada, titulo, autor, género y sinopsis. El usuario decide volver, iniciar sesión o continuar leyendo según permisos.
Flujos alternativos	Si la obra deja de ser publica, el sistema informa que ya no está disponible.
Postcondiciones	El usuario conoce la descripción general de la obra.

Casos relacionados	Extiende CU-01 y puede conducir a CU-13.
Nivel de análisis	descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-03: Iniciar Sesión / Registrarse

Actor principal	Visitante
Objetivo	Obtener una identidad válida para acceder a funciones restringidas.
Precondiciones	El usuario no tiene sesión activa.
Flujo básico	El usuario elige entrar o registrarse, proporciona sus datos, el sistema valida la identidad y crea una sesión.
Flujos alternativos	Credenciales incorrectas, correo duplicado o datos inválidos muestran mensajes de corrección.
Postcondiciones	El usuario queda autenticado o recibe una causa clara de rechazo.
Casos relacionados	Incluido por casos que requieren identidad.
Nivel de análisis	descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-04: Gestionar Perfil

Actor principal	Usuario registrado
Objetivo	Mantener actualizada la información personal de la cuenta.
Precondiciones	El usuario está autenticado.
Flujo básico	El usuario abre configuración, revisa sus datos, modifica campos permitidos y guarda cambios.
Flujos alternativos	Datos inválidos o sesión expirada impiden guardar y piden corrección o nuevo inicio de sesión.
Postcondiciones	El perfil queda actualizado para los clientes compatibles.
Casos relacionados	Requiere CU-03.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-05: Crear Proyecto Narrativo

Actor principal	Escritor
Objetivo	Crear un espacio de trabajo para una obra o universo narrativo.

Precondiciones	El escritor esta autenticado.
Flujo básico	El escritor solicita crear proyecto, captura título, genero, sinopsis y visibilidad, confirma y el sistema prepara el nuevo espacio.
Flujos alternativos	Datos obligatorios faltantes o inválidos detienen la creación.
Postcondiciones	El proyecto existe y el creador queda como propietario.
Casos relacionados	Incluye CU-03; se relaciona con CU-07 y CU-08.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-06: Gestionar Colaboradores

Actor principal	Escritor propietario
Objetivo	Controlar quienes participan en un proyecto y con que permisos.
Precondiciones	El proyecto existe y el usuario es propietario.
Flujo básico	El propietario revisa colaboradores, invita usuarios, asigna roles y puede retirar accesos.
Flujos alternativos	Usuario inexistente, rol invalido o intento de retirar al propietario son rechazados.
Postcondiciones	La lista de colaboradores refleja los permisos actuales.
Casos relacionados	Requiere CU-05; afecta CU-08, CU-09, CU-10 y CU-11.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-07: Configurar Privacidad

Actor principal	Escritor propietario
Objetivo	Definir si una obra es privada o pública para lectores.
Precondiciones	El proyecto existe y el usuario es propietario.
Flujo básico	El propietario abre la configuración, cambia la visibilidad y el sistema actualiza el acceso público.
Flujos alternativos	Si la obra no cumple condiciones mínimas de publicación, el sistema advierte o bloquea el cambio.
Postcondiciones	La visibilidad del proyecto queda actualizada.
Casos relacionados	Extiende CU-05; afecta CU-01, CU-02 y CU-13.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-08: Editar Manuscrito

Actor principal	Escritor / Editor
Objetivo	Redactar y mantener capítulos con apoyo colaborativo.
Precondiciones	El usuario tiene permiso de escritura en el proyecto.
Flujo básico	El usuario abre un capítulo, edita contenido enriquecido, guarda avances y observa actividad de colaboradores conectados.
Flujos alternativos	Si hay pérdida de conexión o conflicto de cambios, el sistema conserva el avance posible e informa el estado.
Postcondiciones	El capítulo conserva la versión más reciente disponible.
Casos relacionados	Requiere CU-05 y permisos de CU-06.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-09: Gestionar Wiki

Actor principal	Escritor / Editor
Objetivo	Mantener entidades del universo narrativo como personajes, lugares, objetos o conceptos.
Precondiciones	El usuario tiene permiso de edición en el proyecto.
Flujo básico	El usuario consulta entidades, crea o actualiza fichas, clasifica tipos y registra información relevante.
Flujos alternativos	Datos inválidos o eliminación de entidades relacionadas requieren corrección o confirmación.
Postcondiciones	La wiki del proyecto queda actualizada.
Casos relacionados	Alimenta CU-10 y apoya CU-08.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-10: Visualizar Grafo Narrativo

Actor principal	Escritor / Editor / Lector autorizado
Objetivo	Comprender relaciones entre entidades del universo narrativo.
Precondiciones	El usuario tiene acceso al proyecto.
Flujo básico	El usuario abre el grafo, explora nodos y relaciones, y según permisos puede crear o retirar relaciones.
Flujos alternativos	Si no existen entidades, el sistema muestra un grafo vacío y orienta al usuario.
Postcondiciones	El usuario visualiza o actualiza relaciones narrativas.

Casos relacionados	Usa información de CU-09.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-11: Participar en Voz como Hablante

Actor principal	Escritor / Editor
Objetivo	Coordinar la colaboración mediante comunicación push-to-talk.
Precondiciones	El usuario esta autenticado, tiene acceso al proyecto y dispone de micrófono.
Flujo básico	El usuario entra a la sala, el sistema muestra participantes, el usuario activa hablar y el audio se transmite a la sala.
Flujos alternativos	Si no hay micrófono o permiso de audio, el sistema permite permanecer sin transmitir.
Postcondiciones	Los participantes reciben el audio durante la intervención.
Casos relacionados	Complementa CU-08.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-12: Unirse a Voz como Oyente

Actor principal	Lector registrado / Colaborador
Objetivo	Escuchar una sesión de voz del proyecto sin transmitir audio.
Precondiciones	El usuario tiene acceso al proyecto y la sala esta disponible.
Flujo básico	El usuario se une a la sala, escucha participantes activos y puede abandonar cuando lo decida.
Flujos alternativos	Si no hay participantes, el sistema permite esperar o salir.
Postcondiciones	El usuario estuvo conectado como oyente.
Casos relacionados	Relacionado con CU-11.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-13: Leer Historia Publicada

Actor principal	Lector / Visitante según visibilidad
Objetivo	Consumir capítulos de una obra publica o autorizada.

Precondiciones	La obra esta publica o el usuario tiene permiso de lectura.
Flujo básico	El usuario selecciona una obra, revisa capítulos disponibles, abre un capítulo y navega el contenido.
Flujos alternativos	Si el acceso cambia a privado, el sistema bloquea contenido no autorizado.
Postcondiciones	El usuario lee el contenido disponible sin modificarlo.
Casos relacionados	Extiende CU-01 y CU-02.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-14: Consultar Reportes del Sistema

Actor principal	Administrador
Objetivo	Conocer métricas generales de uso y estado de la plataforma.
Precondiciones	El administrador esta autenticado.
Flujo básico	El administrador abre el panel de reportes, revisa métricas y actualiza la consulta según necesidad.
Flujos alternativos	Si no existen datos o el usuario no es Admin, se muestra estado vacío o acceso denegado.
Postcondiciones	El administrador obtiene una vista del estado operativo.
Casos relacionados	Requiere CU-03.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

CU-15: Gestionar Usuarios

Actor principal	Administrador
Objetivo	Supervisar cuentas y aplicar acciones administrativas.
Precondiciones	El administrador esta autenticado y autorizado.
Flujo básico	El administrador consulta usuarios, busca una cuenta y aplica acciones permitidas como bloqueo o eliminación.
Flujos alternativos	El sistema impide acciones sobre la propia cuenta o sobre usuarios no encontrados.
Postcondiciones	La cuenta objetivo queda en el estado administrativo indicado.
Casos relacionados	Puede invalidar sesiones asociadas a CU-03.
Nivel de análisis	Descripción conceptual. Los detalles técnicos, endpoints y componentes se documentan en la sección de diseño.

Prototipo de UI

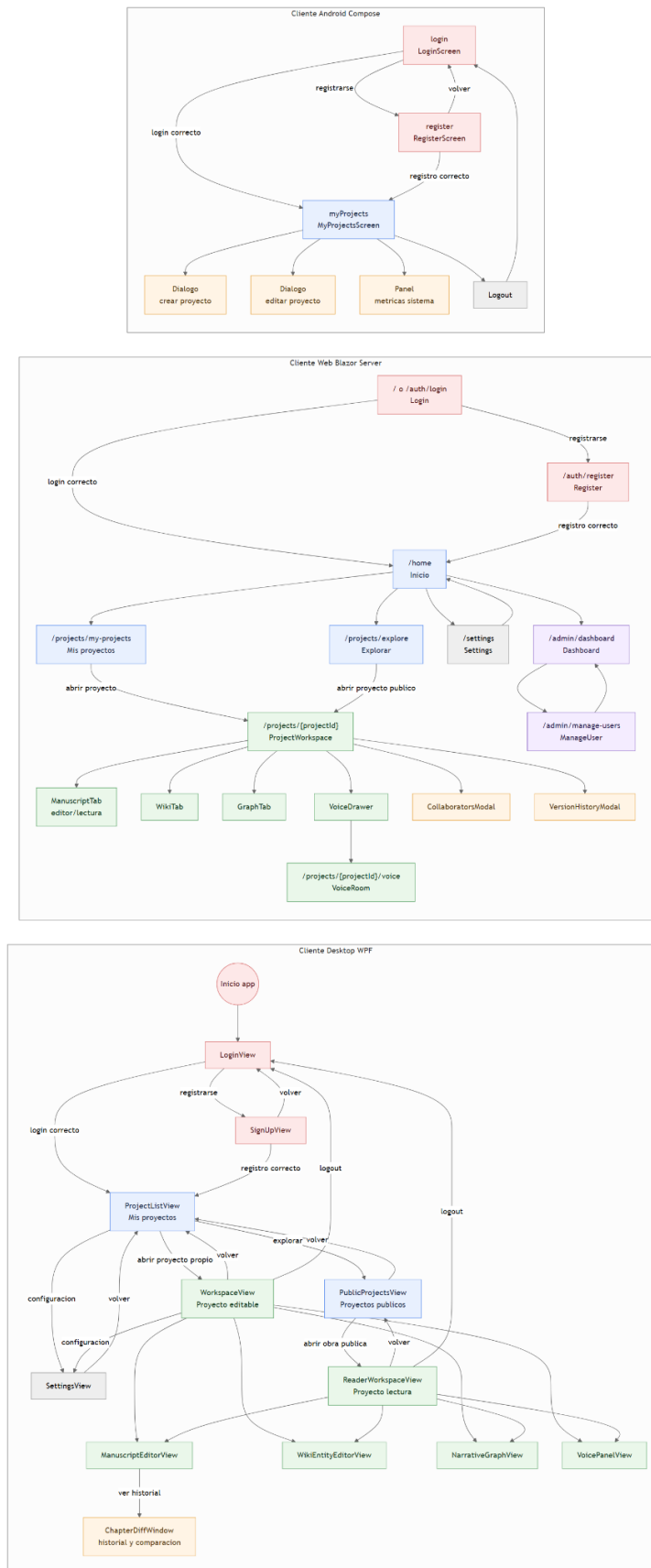
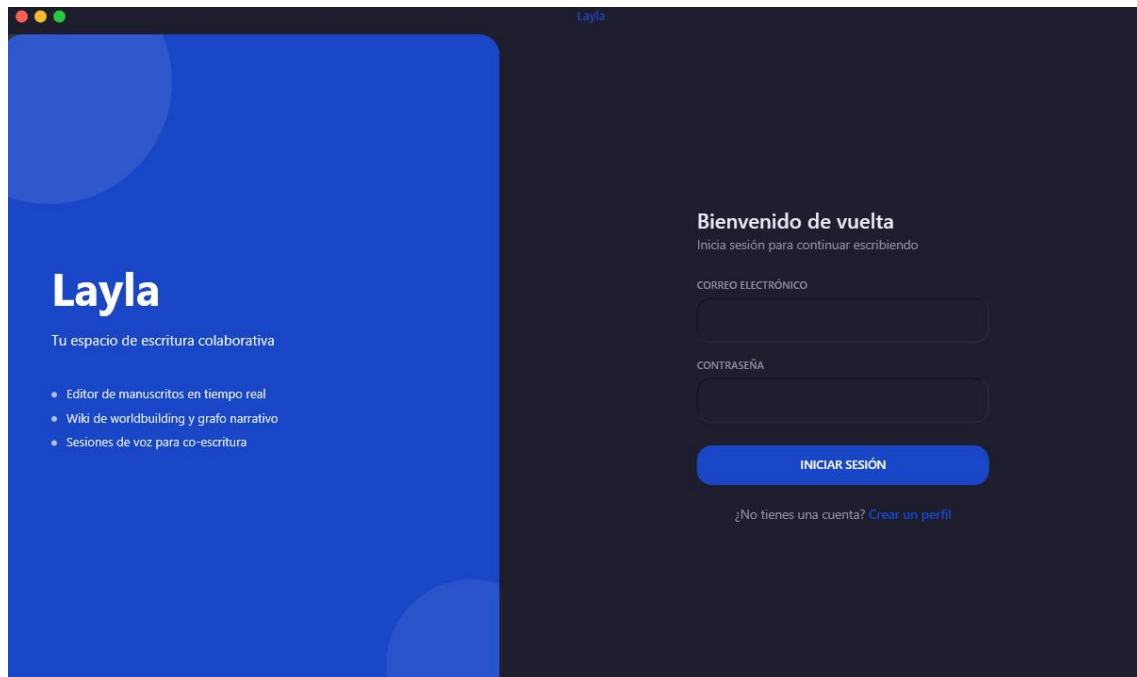


Diagrama 2 - Flujos de vistas implementados.

Cliente de Escritorio

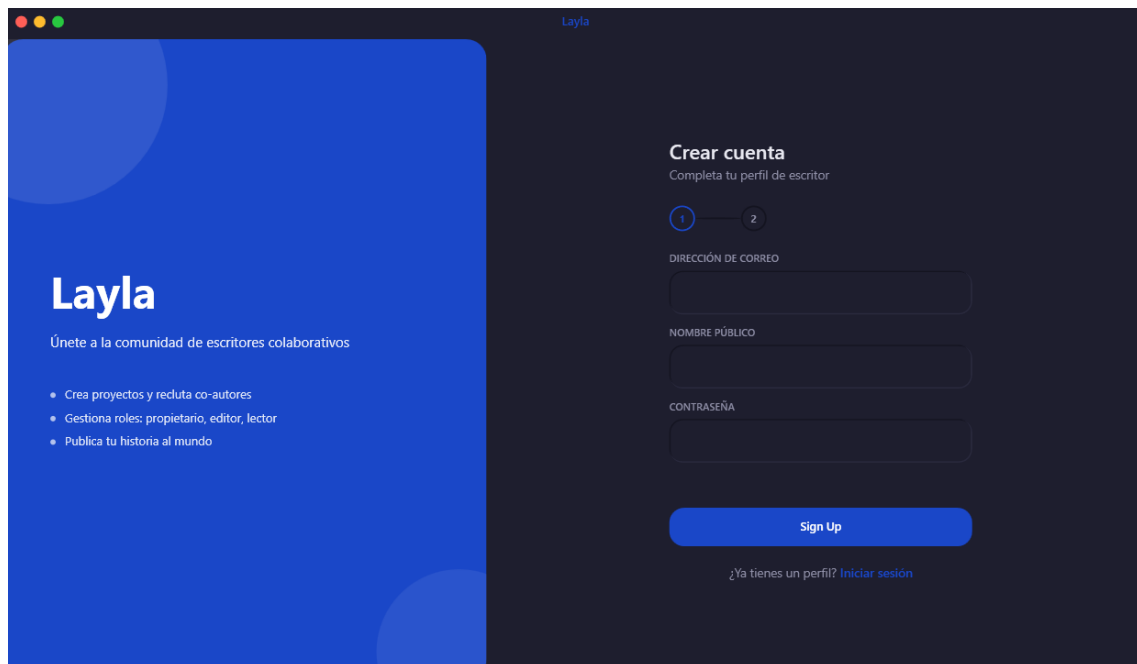
Login



The login screen for the Layla desktop client features a dark blue sidebar on the left with the 'Layla' logo and a list of features: 'Editor de manuscritos en tiempo real', 'Wiki de worldbuilding y grafo narrativo', and 'Sesiones de voz para co-escritura'. The main area is dark navy blue and contains a 'Bienvenido de vuelta' (Welcome back) message, a prompt to 'Inicia sesión para continuar escribiendo', and input fields for 'CORREO ELECTRÓNICO' and 'CONTRASEÑA'. A blue 'INICIAR SESIÓN' button is positioned below the password field, followed by a link to 'Crear un perfil' for users who do not have an account.

Prototipo 1 - Inicio de sesión.

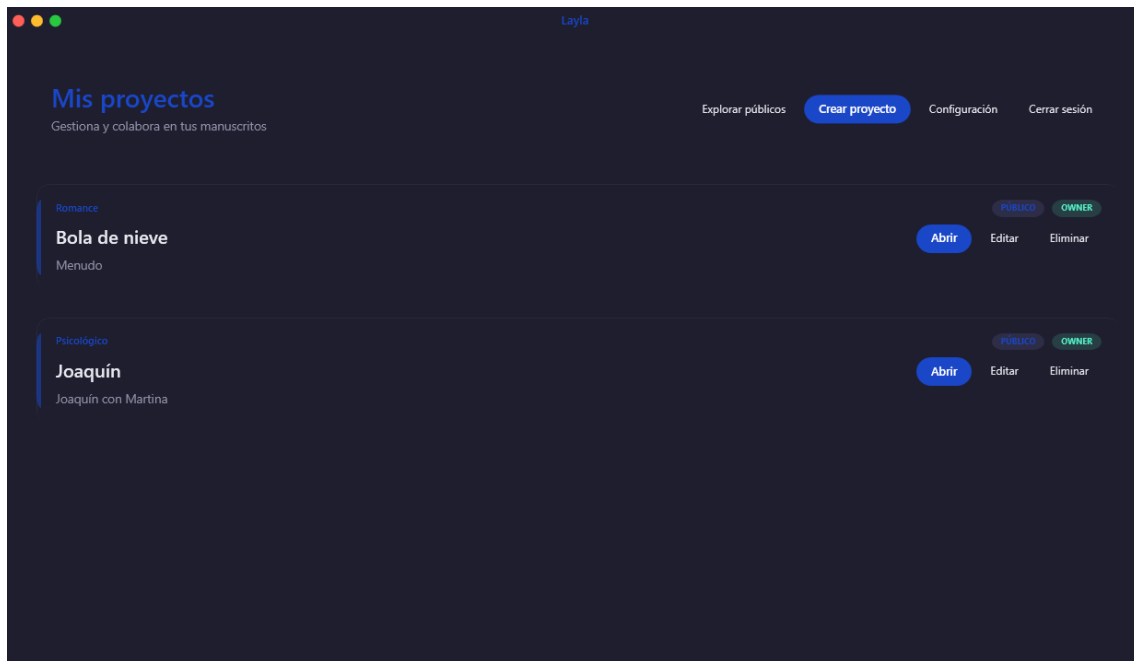
Registro



The registration screen for the Layla desktop client has a similar layout to the login screen. The sidebar on the left promotes joining the 'comunidad de escritores colaborativos' and lists features like 'Crea proyectos y recluta co-autores', 'Gestiona roles: propietario, editor, lector', and 'Publica tu historia al mundo'. The main area is titled 'Crear cuenta' (Create account) and includes a progress indicator for two steps. It contains input fields for 'DIRECCIÓN DE CORREO', 'NOMBRE PÚBLICO', and 'CONTRASEÑA', followed by a blue 'Sign Up' button. A link to 'Iniciar sesión' is provided for users who already have a profile.

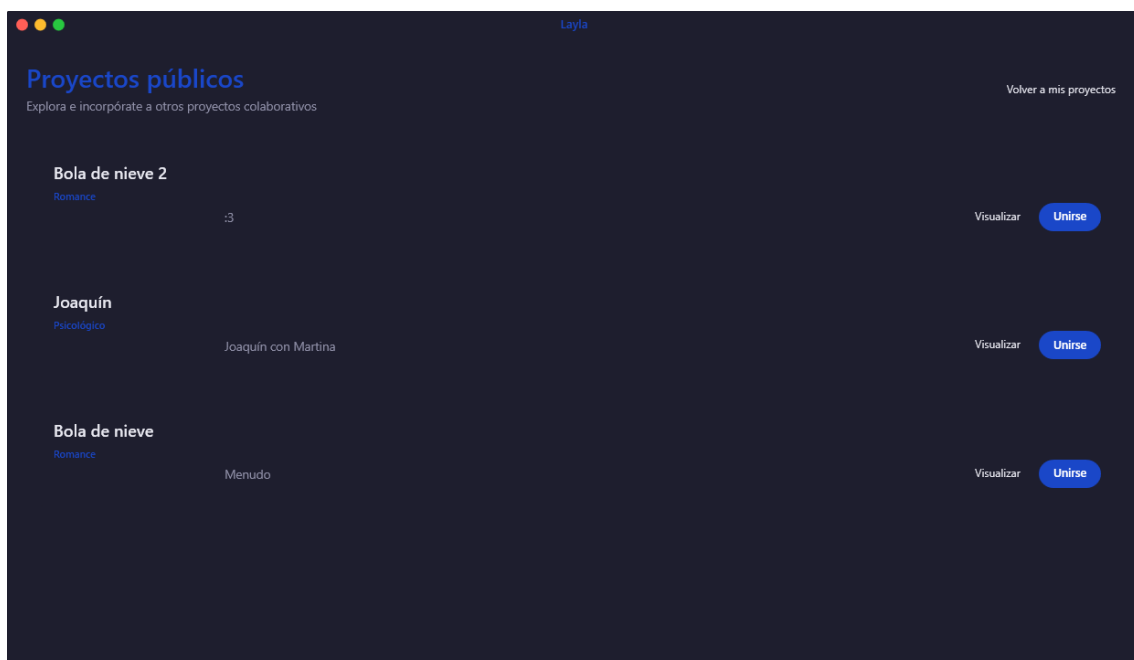
Prototipo 2 - Registro de usuario nuevo.

Proyectos



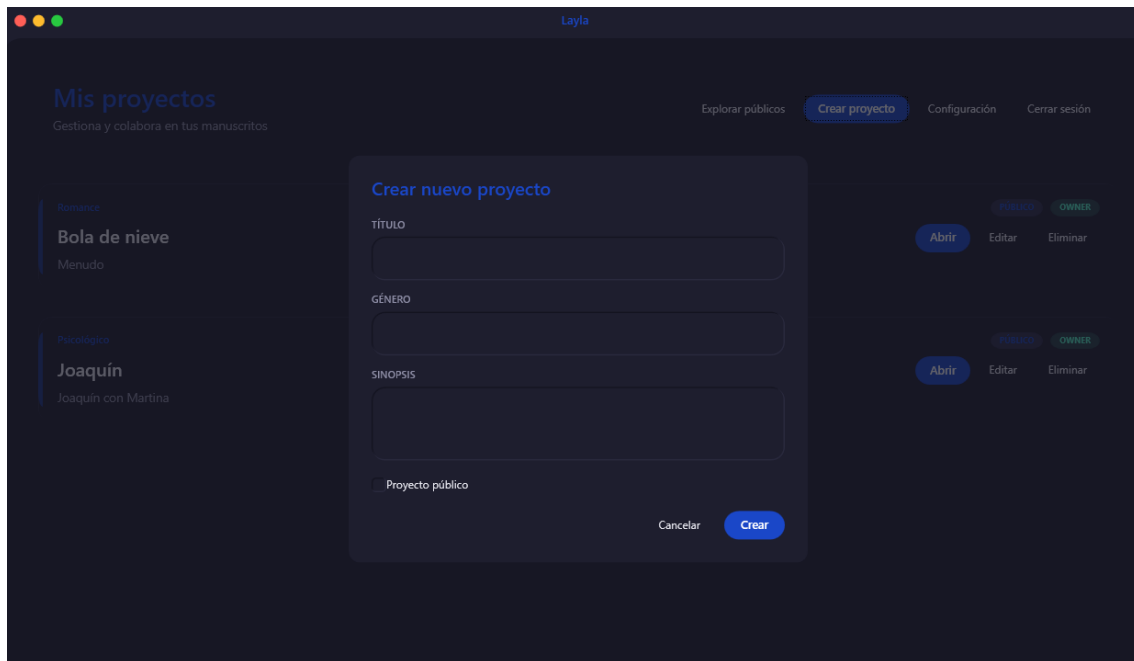
Prototipo 3 - Consulta de "Mis proyectos".

Proyectos públicos



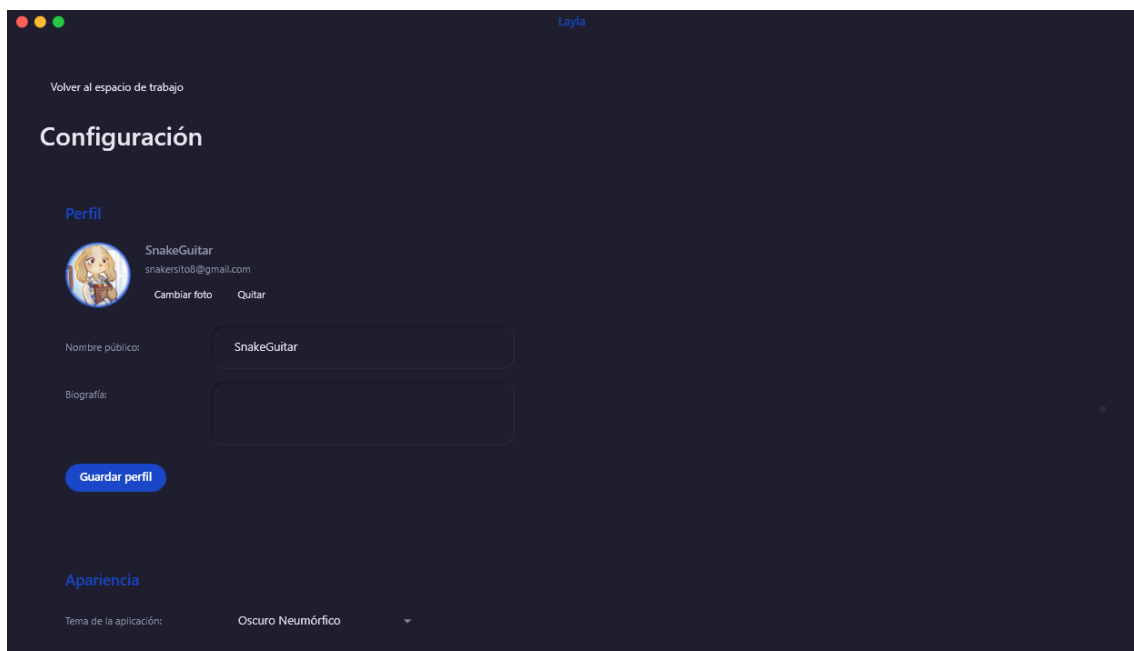
Prototipo 4 - Consulta de proyectos públicos.

Nuevo proyecto



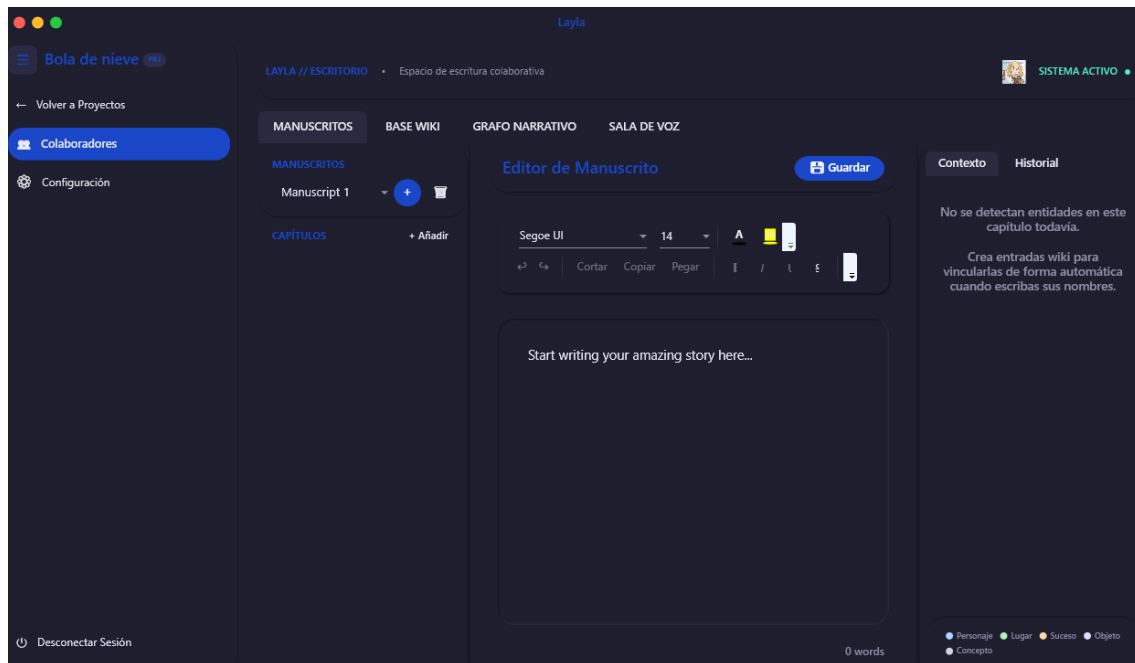
Prototipo 5 - Creación de un nuevo proyecto.

Configuración



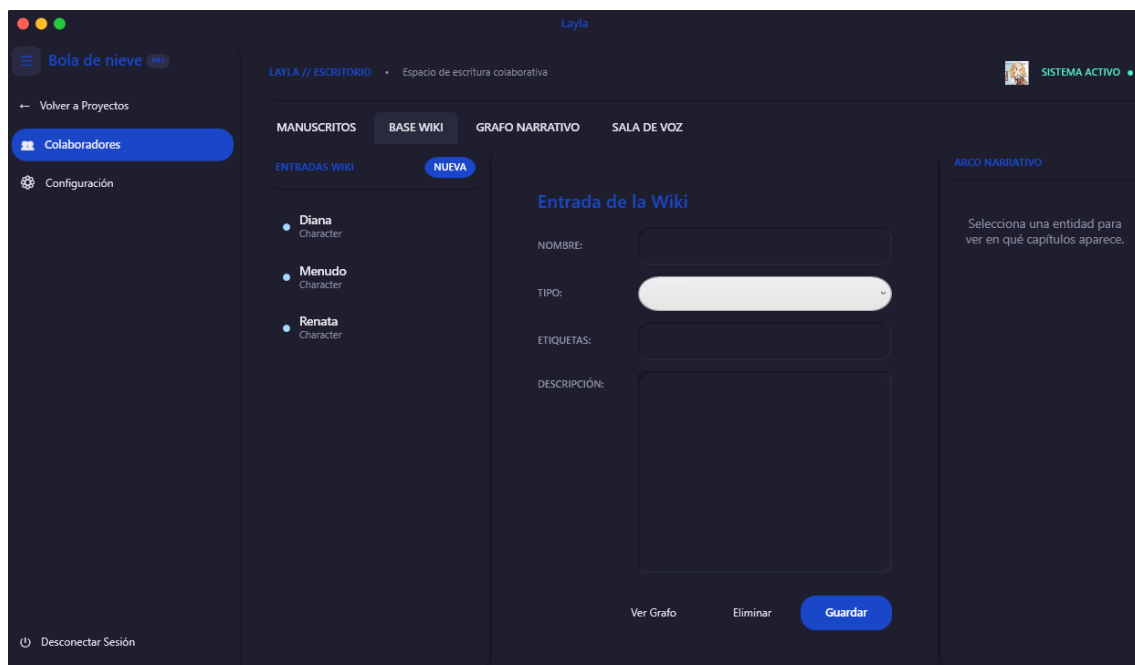
Prototipo 6 - Configuración del perfil propio del usuario.

Espacio de trabajo (manuscritos)



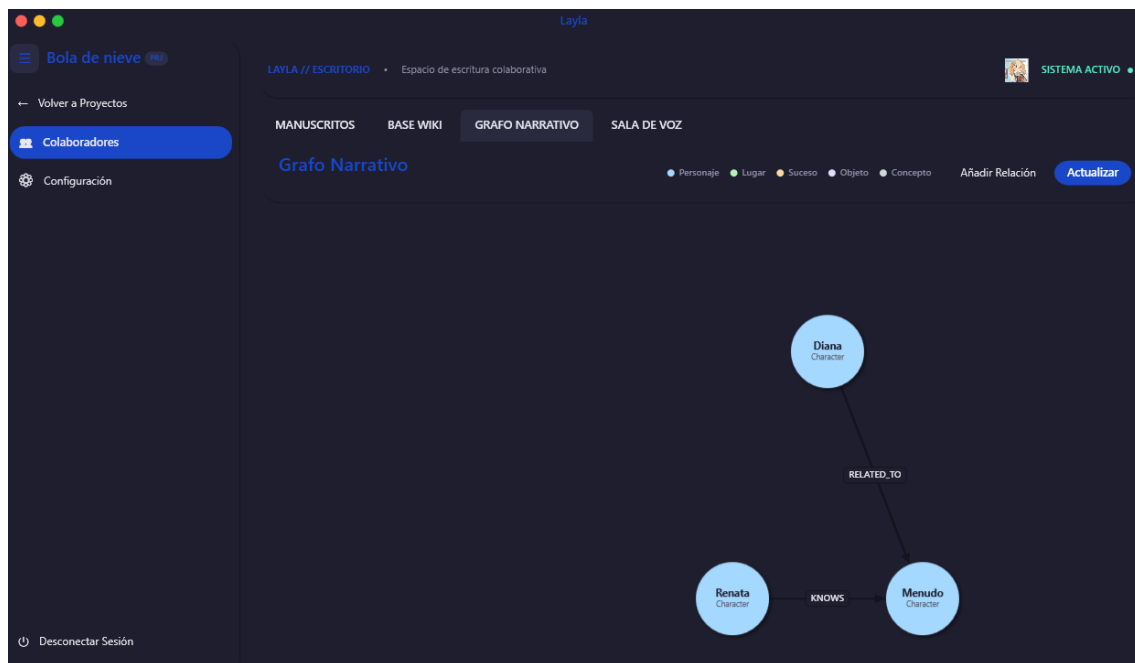
Prototipo 7 - Espacio de trabajo para la creación de manuscritos.

Espacio de trabajo (wiki)



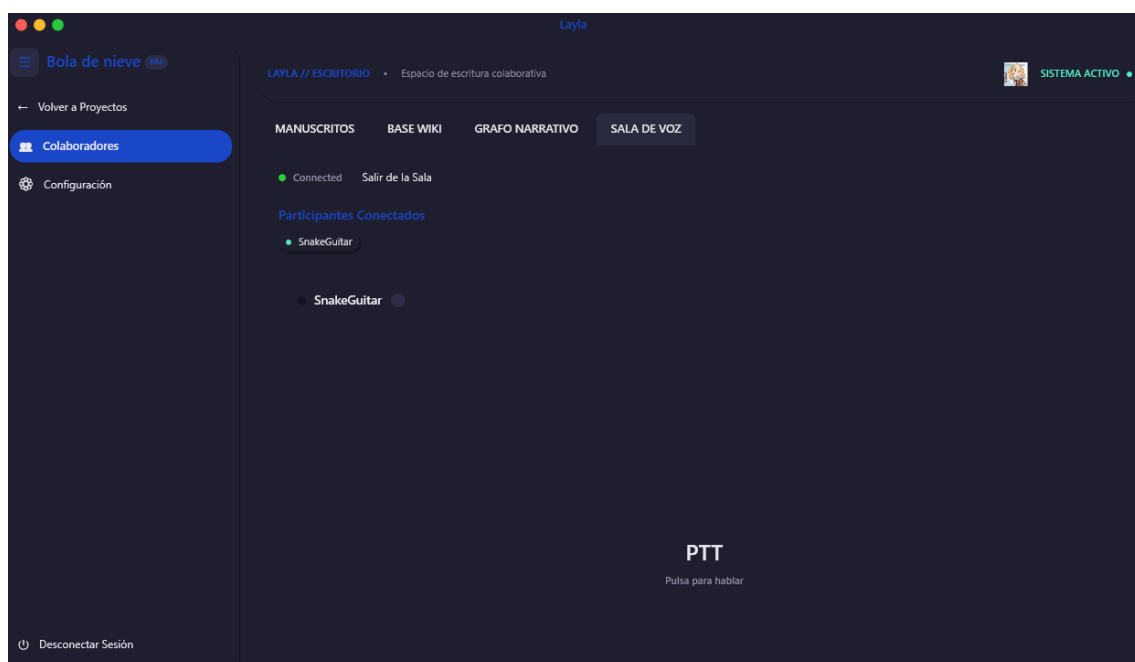
Prototipo 8 – Espacio de trabajo para la creación de wiki.

Espacio de trabajo (grafos)



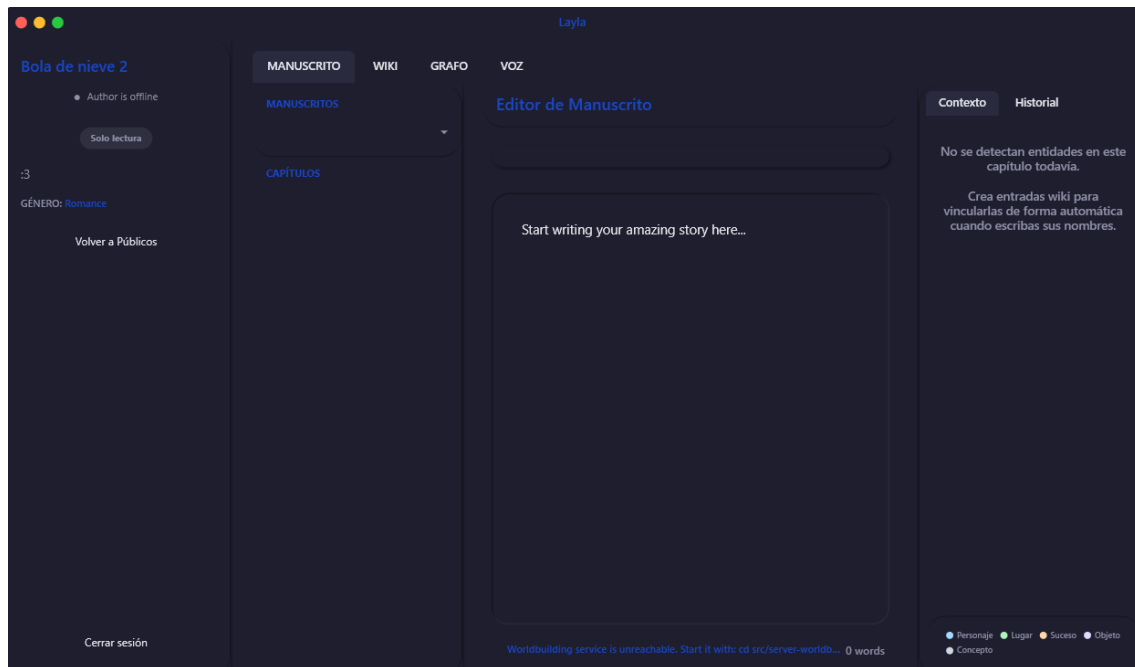
Prototipo 9 - Espacio de trabajo para manejo de grafos.

Espacio de trabajo (sala de voz)



Prototipo 10 - Espacio de trabajo de la sala de voz.

Espacio de trabajo (solo lectura)



Prototipo 11 - Espacio de trabajo del proyecto (solo permiso de lectura).

Estrategia de Implementación de UI (Enfoque MVP Académico)

Dado el alcance temporal del proyecto y los requisitos de las asignaturas ("Desarrollo de Sistemas en Red" y "Despliegue de Software"), se ha definido una estrategia de **Bajo Código y Alta Eficiencia (Low-Code / High-Efficiency)**. El objetivo es priorizar la funcionalidad distribuida y la heterogeneidad sobre la complejidad visual.

A. Análisis del Flujo de Navegación

El diseño de ventanas responde a una **arquitectura de navegación segmentada**, donde cada cliente se especializa en una tarea única para evitar la duplicidad de lógica compleja en el frontend:

1. **Cliente Escritorio (WPF):** Centraliza la carga de trabajo pesado (*Deep Work*). Su navegación se basa en pestañas (TabControl) dentro de un espacio de trabajo único, eliminando la necesidad de múltiples ventanas emergentes.
2. **Cliente Web (Blazor Server):** Centraliza autoría, lectura y administración. Su flujo distingue usuarios estándar, colaboradores y administradores sin abandonar el modelo de cliente web de workspace completo.
3. **Cliente Móvil (Android):** centraliza operaciones ligeras de administración de proyectos y consulta de métricas. Su flujo es lineal y no duplica el workspace creativo.

B. Propuesta de Simplificación de Componentes

Para mitigar riesgos técnicos durante la fase de construcción, se utilizarán controles nativos y estandarizados en lugar de desarrollos personalizados complejos:

1. Cliente Escritorio (WPF) - Estrategia "Controles Nativos"

- **Editor de Texto:** Se utilizará el control nativo `System.Windows.Controls.RichTextBox`. El contenido se persistirá directamente como cadenas XAML/RTF o HTML en MongoDB, evitando la creación de parsers JSON personalizados.
- **Visualización de Grafos:** Se implementará mediante un Canvas estándar de WPF, representando nodos con formas básicas (Ellipse) y relaciones con líneas (Line). No se integrarán librerías de física de grafos de terceros; la visualización será estática para cumplir el requisito funcional sin aumentar la carga de procesamiento.
- **Layout:** Se priorizará el uso de Grid sobre StackPanel para garantizar la escalabilidad de los formularios de la Wiki.

2. Cliente Web (Blazor Server) - Estrategia "Server-Side Rendering"

- **Diseño Visual:** Se implementará una plantilla basada en **Bootstrap 5** para la estructura del Dashboard y el Feed, reduciendo el tiempo de maquetado CSS.
- **Lógica de Interfaz:** Se utilizará **Renderizado en Servidor (SSR)** puro. Las interacciones de navegación (ej. "Leer Capítulo") recargarán la página completa, evitando la complejidad de gestionar estado en el cliente (SPA) con JavaScript/AJAX, lo que simplifica la depuración de errores de red.
- **Visualización de Datos:** Los reportes administrativos se presentarán mediante tarjetas de contadores simples o tablas, reservando el uso de librerías gráficas (como Chart.js) solo si el tiempo de desarrollo lo permite.

3. Cliente Móvil (Android) - Estrategia "Actividades Lineales"

- **Arquitectura UI:** Se utilizarán **Activities** nativas estándar en lugar de una arquitectura de *Single Activity* con *Fragments*, facilitando la gestión del ciclo de vida y la depuración.
- **Control PTT (Push-to-Talk):** La interfaz de voz se simplificará a un único botón (Button) con un `OnTouchListener`. La retroalimentación visual se limitará a cambios de color de estado (Rojo: Transmitiendo / Gris: Inactivo), evitando animaciones de ondas de audio complejas.

Requisitos funcionales

Módulo de Gestión de Usuarios y Seguridad (Backend .NET + SQL Server)

- **RF-01: Autenticación y Autorización.** El sistema debe permitir el registro e inicio de sesión de usuarios, generando un Token (JWT) que identifique su rol (Escritor, Editor, Lector, Admin) y permisos de sesión.
- **RF-02: Gestión de Perfiles.** El sistema debe permitir a los usuarios actualizar su información básica y gestionar sus credenciales de acceso almacenadas en SQL Server.
- **RF-03: Control de Acceso Basado en Roles (RBAC).** El sistema debe restringir el acceso a los módulos de edición y voz según la relación del usuario con el proyecto (ej. un Lector no puede editar; un Editor no puede borrar la obra).

Módulo de Escritura y Proyectos (Backend Node.js + MongoDB)

- **RF-04: Gestión de Proyectos Narrativos.** El sistema debe permitir al Escritor crear, editar y eliminar proyectos, definiendo metadatos como título, sinopsis y género.
- **RF-05: Edición de Texto (Cliente WPF).** El cliente de Escritorio debe implementar un editor de texto enriquecido (Rich Text) basado en **WPF**, capaz de manejar documentos extensos con formato (negritas, cursivas) y renderizado acelerado por hardware, garantizando una experiencia de escritura fluida superior a la de un navegador web.
- **RF-06: Persistencia Documental.** El sistema debe almacenar el contenido de los manuscritos y escaletas en una base de datos documental (MongoDB) para soportar estructuras flexibles y de gran volumen.
- **RF-07: Consistencia de Datos y Dispositivos.** Los clientes de escritorio y web comparten las funciones de autoría sobre manuscritos, wiki y grafo; Android queda fuera del flujo de manuscritos para evitar duplicar una superficie de edición móvil incompleta.

Módulo de Worldbuilding y Relaciones (Backend Node.js + Neo4j)

- **RF-08: Gestión de la Wiki del Mundo.** El sistema debe permitir la creación de fichas de personajes, lugares y objetos, almacenando sus atributos en nodos de una base de datos orientada a grafos (Neo4j).
- **RF-09: Vinculación de Entidades.** El sistema debe permitir establecer y visualizar relaciones semánticas entre nodos (ej. "Personaje A es *hermano* de Personaje B") para generar el grafo de la historia.

Módulo de Colaboración y Voz (Desktop/Web + SignalR)

- **RF-10: Comunicación por Voz (Push-to-Talk).** El sistema implementará comunicación de voz en salas de proyecto mediante SignalR/WebSockets en los clientes de escritorio y web, sin convertir Android en cliente de voz.
- **RF-11: Roles de Voz.** El sistema distinguirá entre roles de "Emisor" (Escritor/Editor activo) y "Oyente" (Lectores/Invitados), gestionando los canales de audio mediante ASP.NET Core SignalR.
- **RF-12: Notificaciones de Presencia.** El sistema debe indicar visualmente qué usuarios están conectados y editando o hablando dentro de un proyecto activo.

Módulo de Lectura y Acceso Público (Web Blazor Server)

- **RF-13: Visualización de Contenido Público.** El portal Web debe permitir a usuarios invitados (Guest) visualizar la sinopsis y fragmentos de obras marcadas como públicas sin necesidad de autenticación.
- **RF-14: Lectura Inmersiva.** El sistema debe presentar al Lector registrado una interfaz limpia de lectura del manuscrito completo, con acceso lateral a la Wiki del mundo.

Módulo de Administración y Reportes (Web Blazor Server)

- **RF-15: Monitoreo del Sistema.** El sistema debe proveer al Administrador un tablero con el estado de los contenedores Docker y la salud de los servicios.
- **RF-16: Generación de Reportes.** El sistema debe generar gráficas estadísticas sobre el uso de la plataforma (número de proyectos, usuarios activos, volumen de datos) para la toma de decisiones.
- **RF-17: Catálogo de Obras Públicas (Feed).** El sistema debe presentar una vista de "Descubrimiento" en el portal web (Blazor Server) que liste las portadas y sinopsis de los proyectos marcados como públicos. Esta lista debe permitir el filtrado por género (ej. Fantasía, Sci-Fi) y ordenamiento por "Más Recientes".
- **RF-18: Sincronización Híbrida.** El cliente WPF debe ser capaz de operar con caché local, permitiendo al escritor continuar redactando en caso de micro cortes de red y sincronizando los cambios con el Backend (.NET) automáticamente al restablecerse la conexión.
 - **Criterio de Aceptación:** Para resolver conflictos de edición simultánea o desfasada (offline), el sistema implementará la política de "**Última Escritura Gana**" (**Last Write Wins - LWW**). Se utilizará la marca de tiempo del servidor (Server-Side Timestamp) al momento

de recibir la petición para determinar qué versión del manuscrito prevalece, descartando versiones anteriores que lleguen con retraso.

- **RF-19: Procesamiento Asíncrono de Datos.** Las operaciones pesadas de escritura en grafos y documentos extensos deben gestionarse mediante colas de mensajes, asegurando que la interfaz de usuario no se congele esperando la confirmación de la base de datos secundaria.

Requisitos no funcionales

Operabilidad y Despliegue

- **RNF-01: Contenerización Total.** Todos los componentes del servidor (Backend .NET, Backend Node.js, SQL Server, MongoDB, Neo4j) deben estar encapsulados en contenedores **Docker**. El despliegue completo de la infraestructura debe ser ejecutable mediante un único comando de orquestación (ej. docker-compose up).
- **RNF-02: Versionado de Imágenes (Rollback Capability).** El sistema de despliegue debe utilizar etiquetado semántico de imágenes (ej. layla-api:v1.0, layla-api:v1.1). El sistema debe permitir volver a una versión anterior estable (rollback) en menos de 5 minutos simplemente cambiando la etiqueta de la imagen en el archivo de orquestación.
- **RNF-03: Separación de Configuración.** Las configuraciones sensibles (cadenas de conexión, claves JWT, puertos) no deben estar "hardcodeadas" en el código ni en la imagen del contenedor, sino que deben inyectarse mediante **Variables de Entorno** (.env) en tiempo de ejecución. Esto permite actualizar configuraciones sin recompilar ni redistribuir el código.
- **RNF-04: Persistencia de Datos Externa (Volúmenes).** Los datos de las bases de datos (SQL, Mongo, Neo4j) deben almacenarse en **Volúmenes de Docker** (Docker Volumes) independientes de los contenedores de aplicación. Esto garantiza que una actualización o destrucción del contenedor de la base de datos no resulte en pérdida de información.

Mantenibilidad y Eficiencia

- **RNF-05: Tiempo de Recuperación (RTO).** En caso de fallo crítico tras una actualización, el tiempo para restaurar el servicio a la versión anterior (Rollback) no debe exceder los **5 minutos**.
- **RNF-06: Automatización de Build.** El proceso de compilación y creación de imágenes Docker debe estar automatizado mediante scripts o un pipeline de CI (Integración Continua), asegurando que el artefacto desplegable sea idéntico en desarrollo y producción.

- **RNF-07: Hot-Swapping (Deseable).** El sistema debe permitir la actualización de los servicios de backend con un tiempo de inactividad (downtime) mínimo, preferiblemente menor a 1 minuto, mediante el reinicio rápido de contenedores ligeros (**Alpine/Debian Slim**).

Fiabilidad y Resiliencia (Reliability)

- **RNF-08: Resiliencia ante Cortes de Red.** En caso de pérdida de conectividad durante sesiones de colaboración o voz en los clientes de escritorio y web, el sistema debe intentar la reconexión y notificar al usuario cuando no sea posible recuperar la sesión.
- **RNF-09: Recuperación Automática de Servicios (Self-Healing).** Si uno de los contenedores del backend (.NET o Node.js) falla o se cierra inesperadamente por una excepción no controlada, el orquestador (Docker) debe reiniciar el contenedor automáticamente mediante la política de reinicio (restart: on-failure) en menos de 10 segundos.
- **RNF-10: Patrón Circuit Breaker (Base de Datos).** El sistema debe implementar el patrón "Cortocircuito" (Circuit Breaker). Si la conexión con MongoDB o SQL Server falla repetidamente (timeout), el backend debe dejar de enviar peticiones inmediatamente y devolver un error controlado (HTTP 503 Service Unavailable) en lugar de saturar la red o bloquear los hilos del servidor esperando respuesta.

Diseño

Diseño arquitectónico

Vista de casos de uso (Diagrama UML de casos de uso)

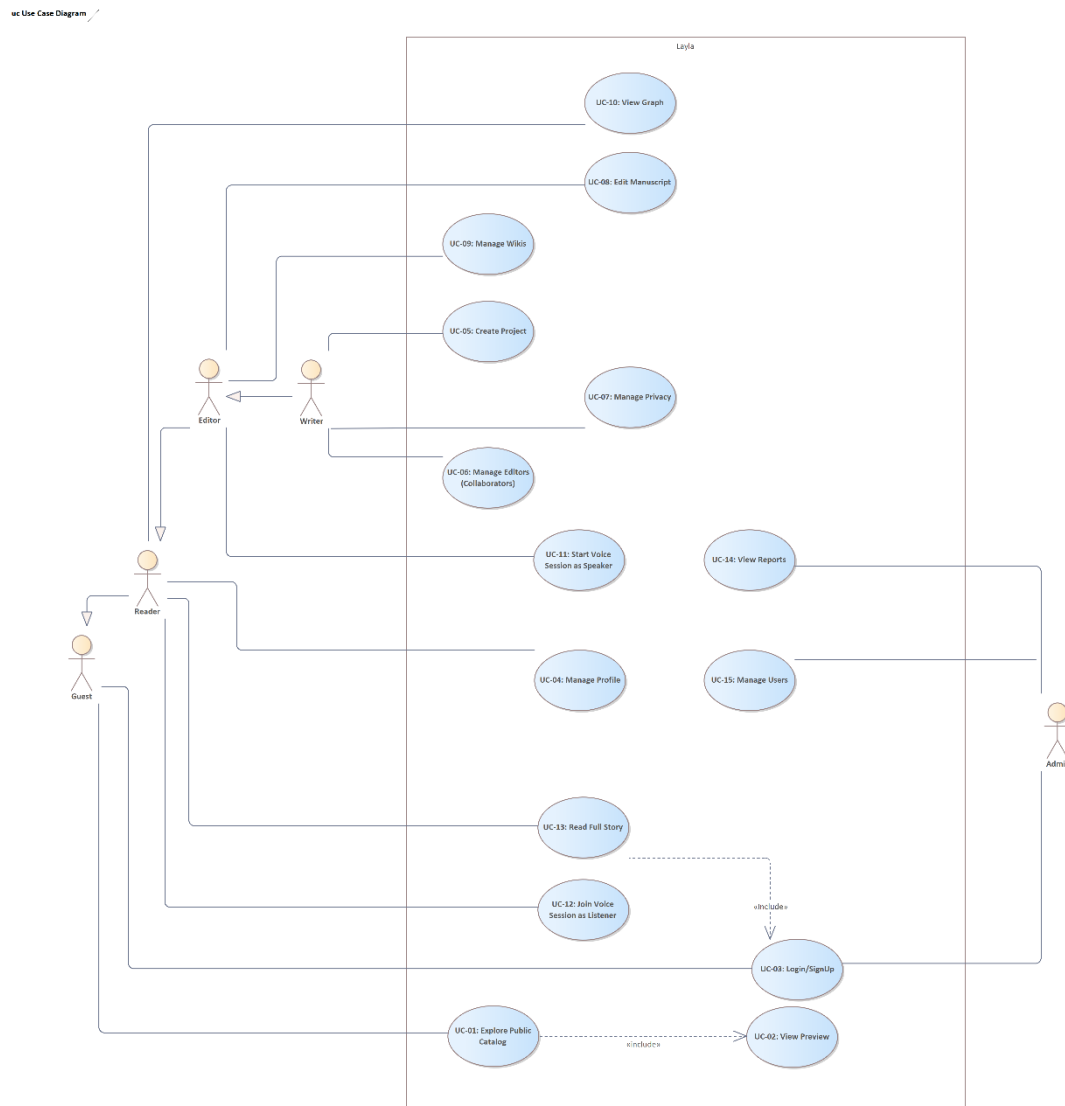


Diagrama 3 - Modelado de casos de uso y actores.

Vista lógica (Diagrama de clases)

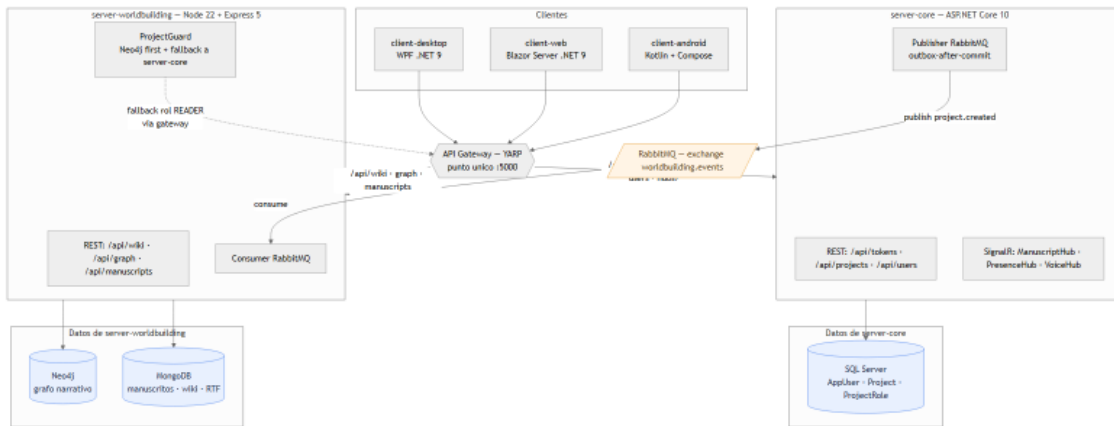


Diagrama 4 - Representación de clases de despliegue virtual.

Vista de implementación (Diagrama de componentes)

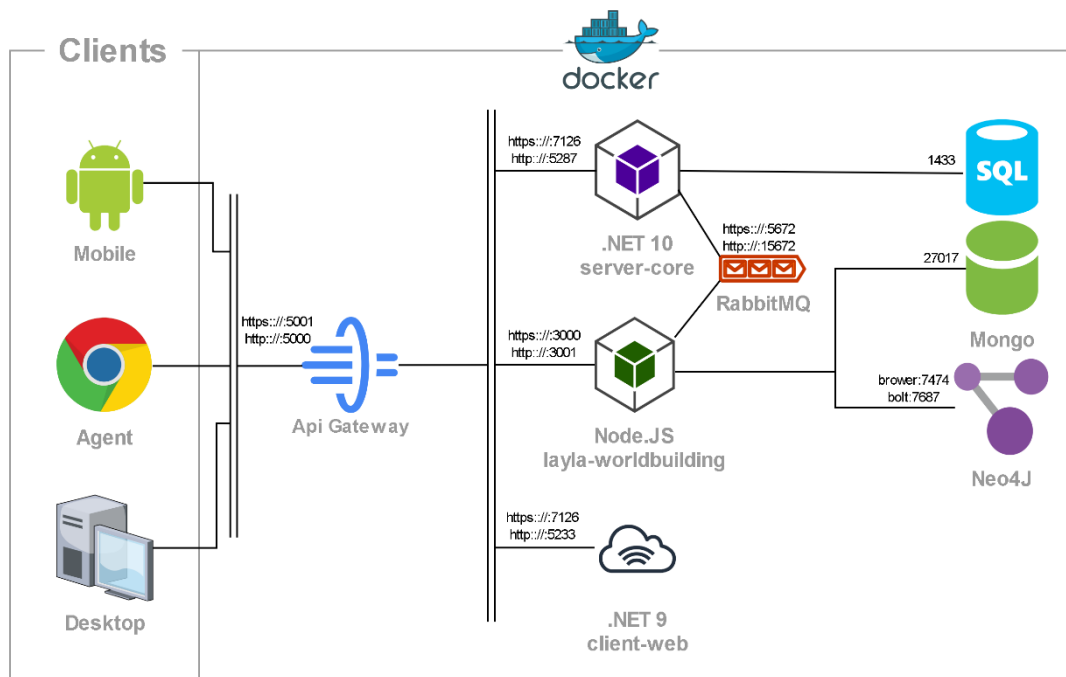


Diagrama 5 - Representación de componentes en implementación.

El diagrama de la Vista de Implementación muestra la arquitectura de servicios y sus dependencias físicas. Corresponde a un diagrama de componentes informal que incluye: los tres clientes (Desktop, Web, Android), el API Gateway (YARP), los dos servicios backend (server-core y server-worldbuilding), el message broker (RabbitMQ), y las tres bases de datos (SQL Server, MongoDB, Neo4j).

Verificación contra el sistema real:

Componente	Tecnología real	Estado en el diagrama
API Gateway	YARP .NET 9 (:5000)	Representado correctamente como punto de entrada único
server-core	ASP.NET Core 10.0	Representado; puertos correctos en configuración dev
server-worldbuilding	Node.js 22 + Express 5	Representado; puerto 3000 correcto
client-web	Blazor Server .NET 9	Representado como componente web desplegado
client-desktop	WPF .NET 9	Representado como cliente Windows
client-android	Kotlin + Compose	Representado como cliente móvil
SQL Server	SQL Server 2022	Representado con relación a server-core
MongoDB	MongoDB 4.4 / 7+	Representado con relación a server-worldbuilding
Neo4j	Neo4j 5 + APOC	Representado con relación a server-worldbuilding
RabbitMQ	RabbitMQ 3 (exchange worldbuilding.events)	Representado entre los dos backends

Vista de procesos

Diagramas de Secuencia

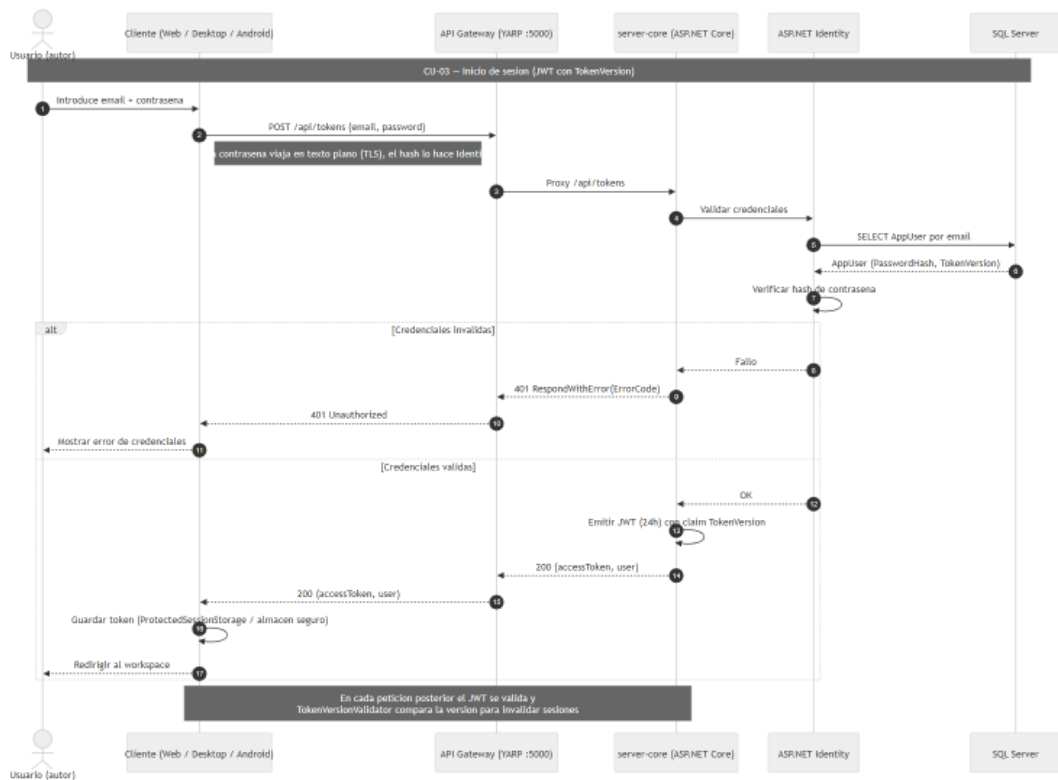


Diagrama 6 - Secuencia: Inicio de sesión con token JWT.

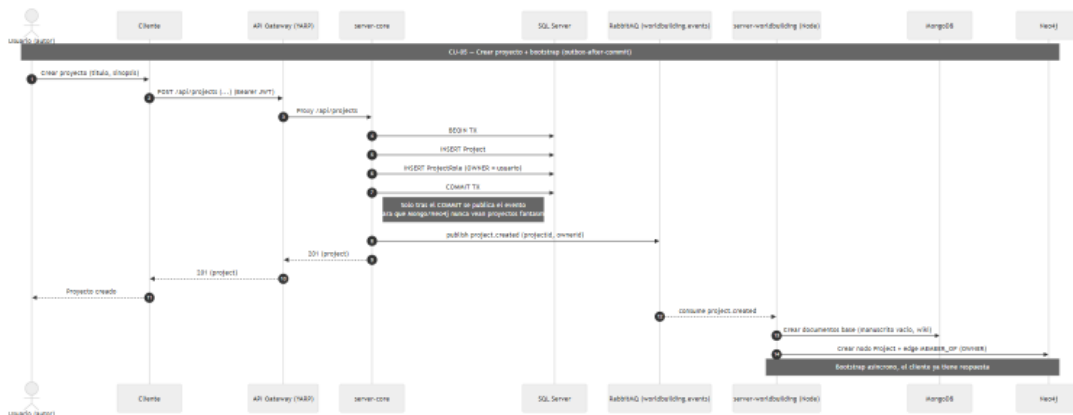


Diagrama 7 - Secuencia: Crear proyecto.

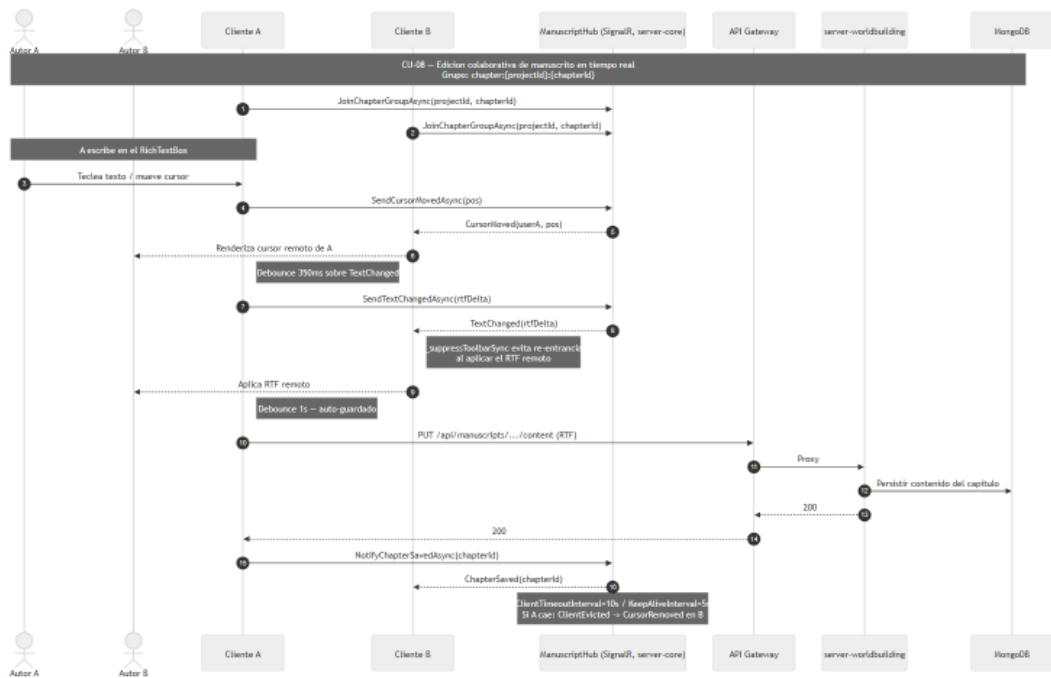
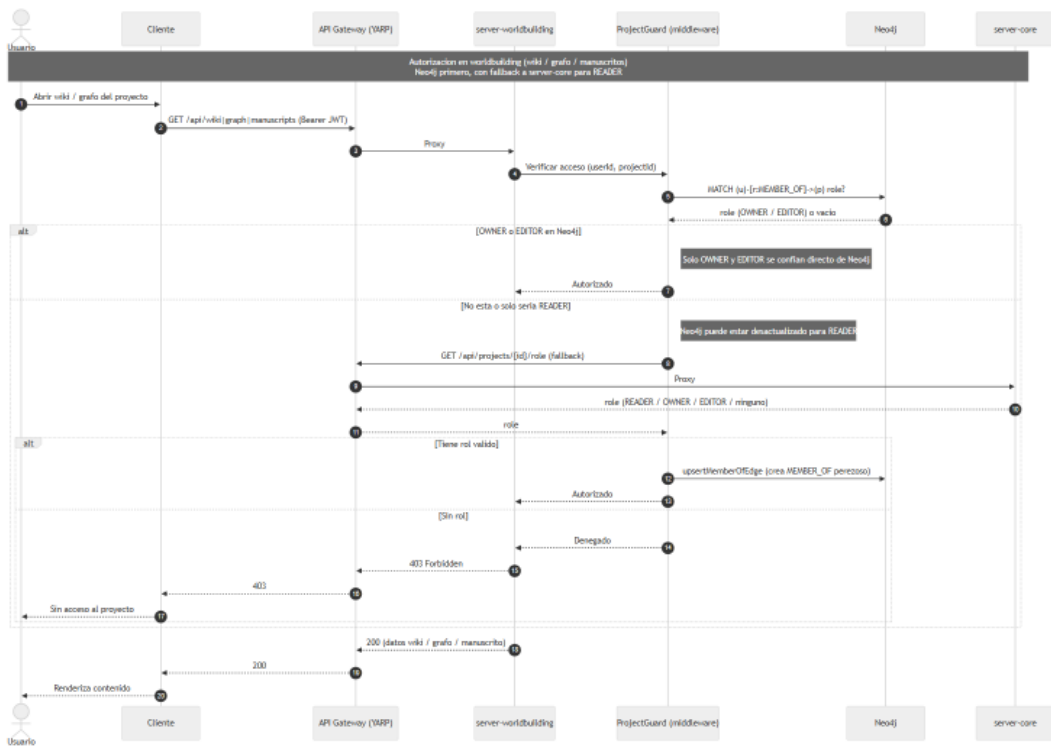


Diagrama 8 - Secuencia: Edición colaborativa de manuscrito en tiempo real.



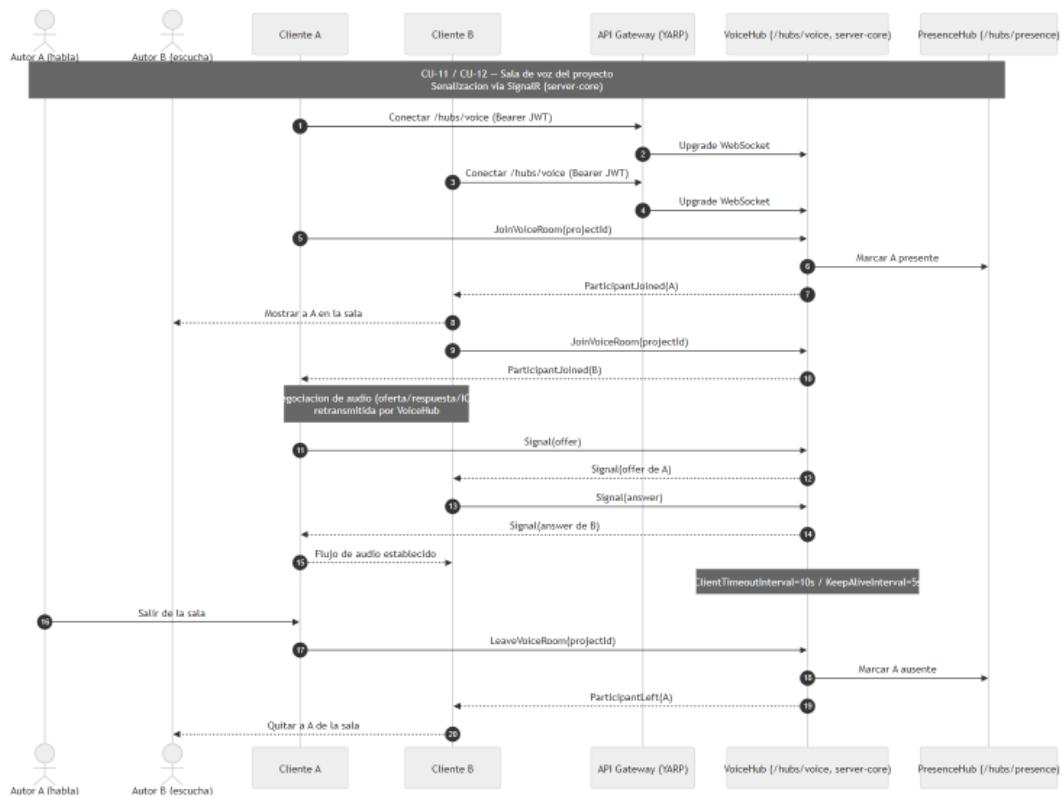


Diagrama 9 - Secuencia: Sala de voz del proyecto.

Diagramas de Estado

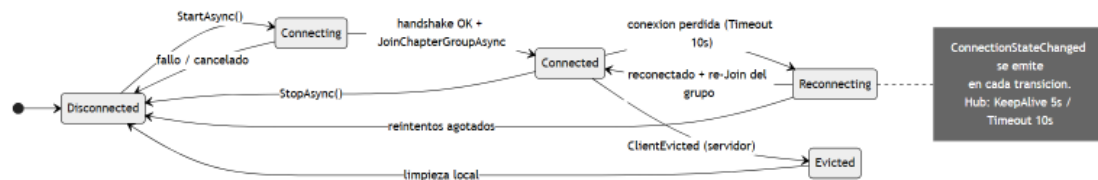


Diagrama 10 - Estado: Conexión a colaboración.

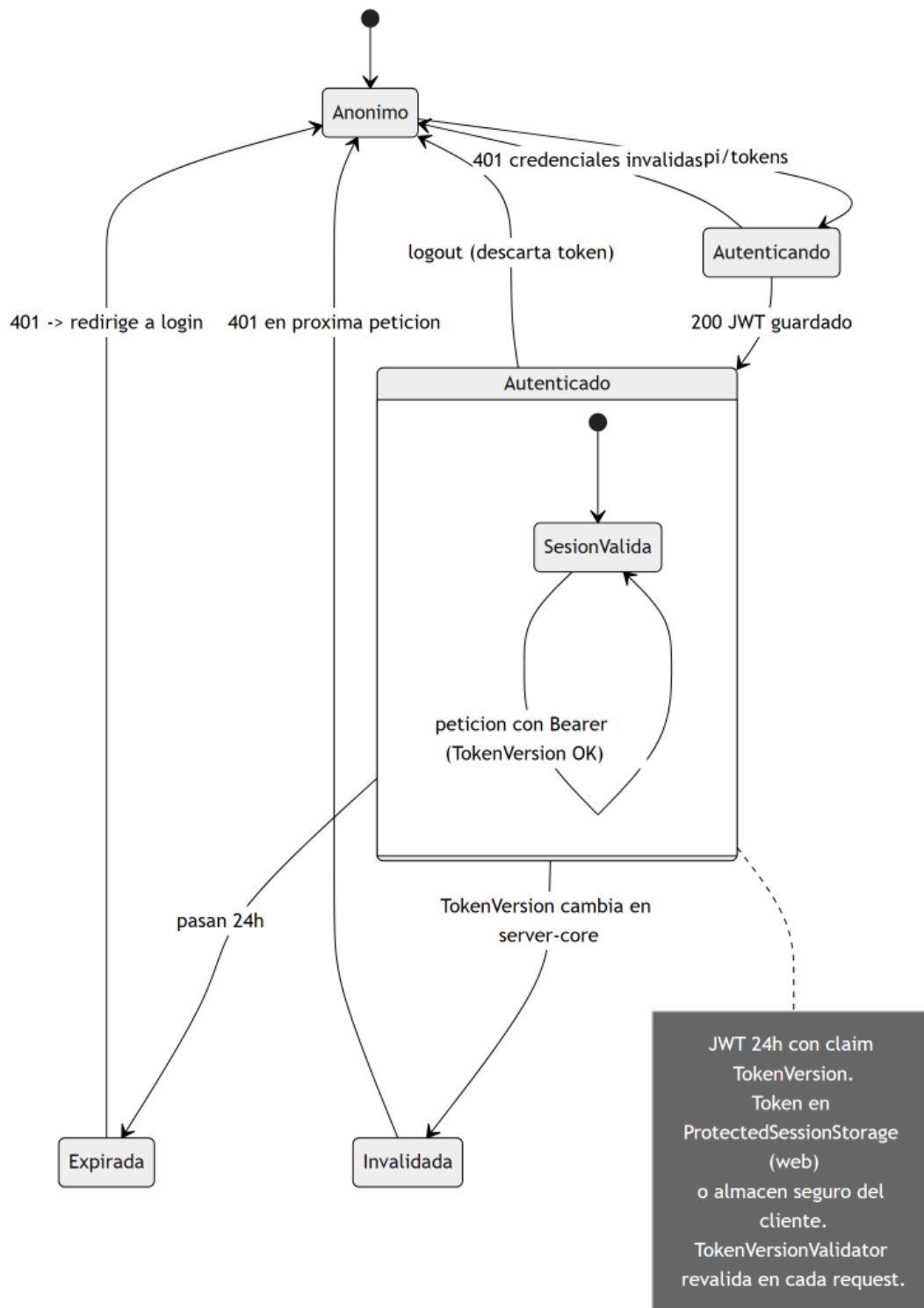


Diagrama 11 - Estado: Autenticación del usuario.

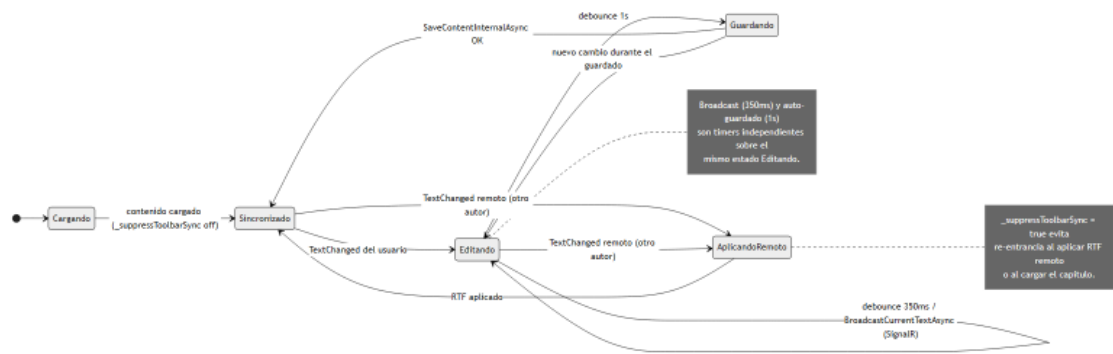


Diagrama 12 - Estado: Edición colaborativa.

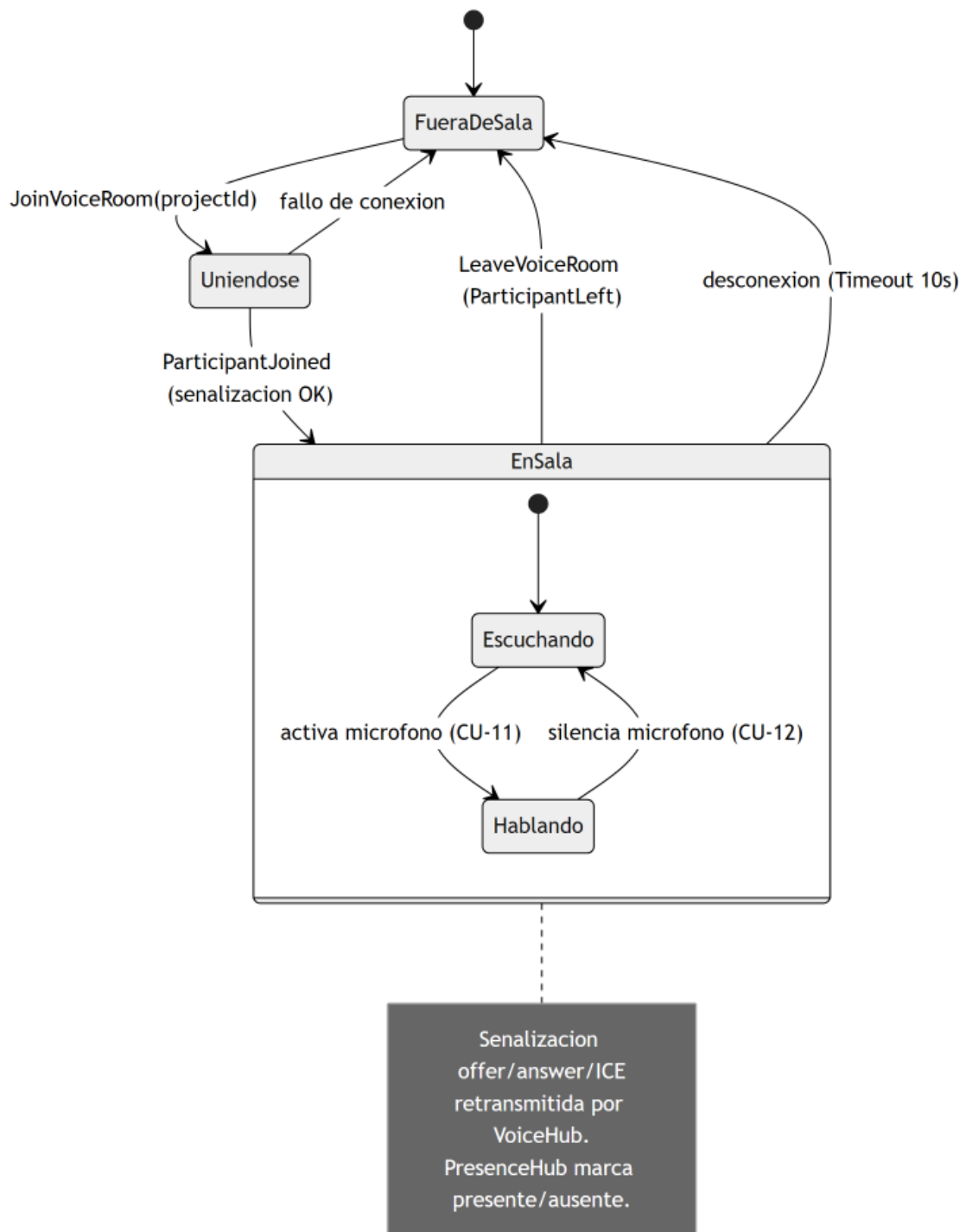


Diagrama 13 - Estado: Estado en sala de voz.

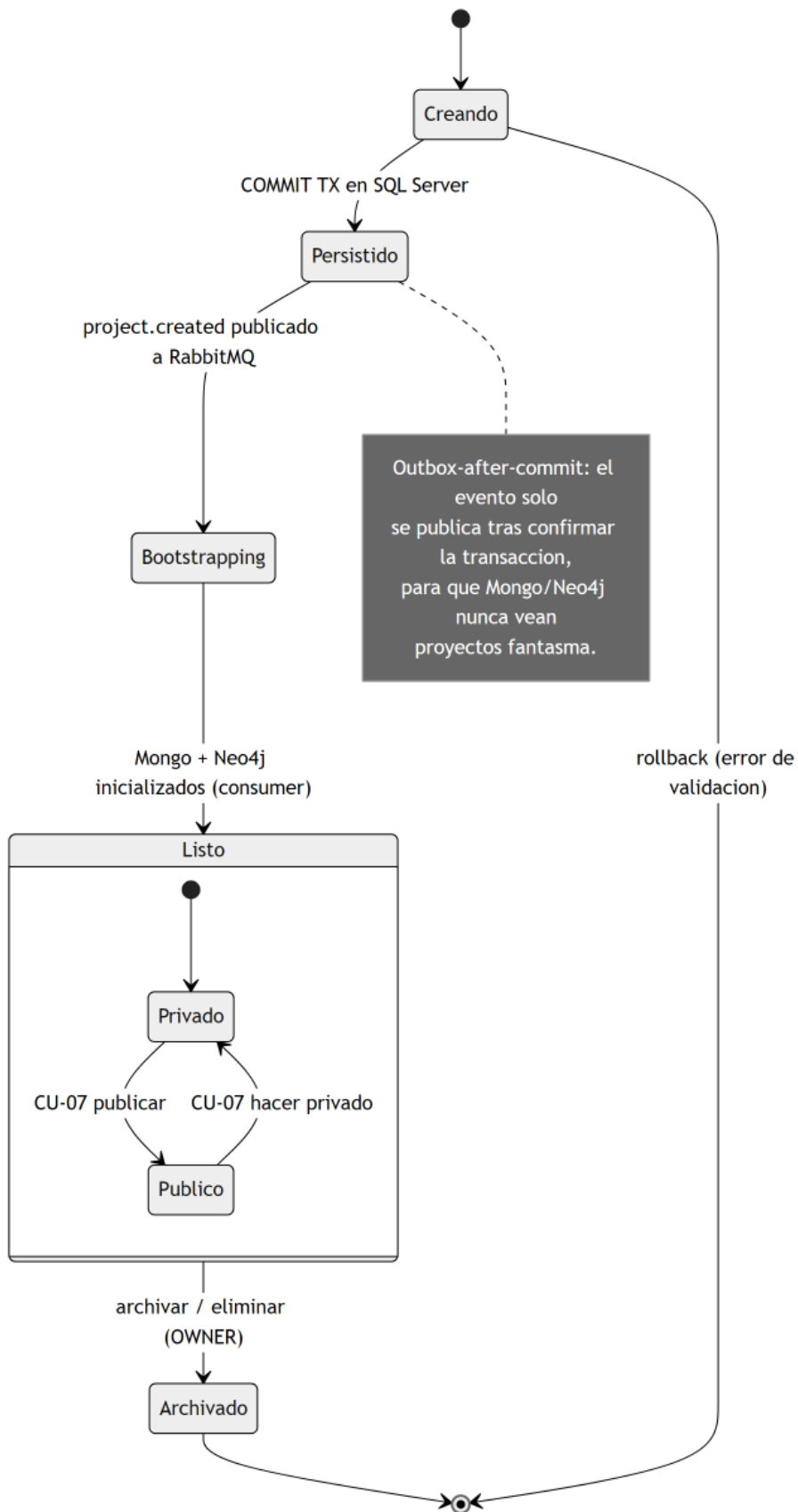


Diagrama 14 - Estado: Cambio de visibilidad del proyecto.

Vista de despliegue (Diagrama de despliegue)

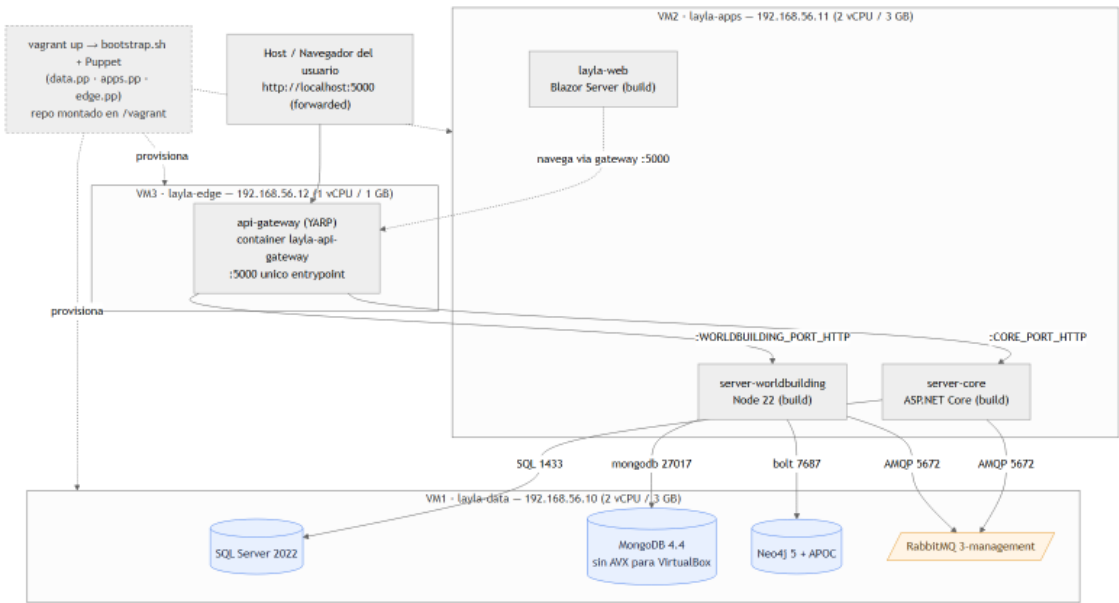


Diagrama 15 - Despliegue: Exposición de los protocolos y puertos.

Control de Acceso y Seguridad (Matriz de roles)

Para garantizar la integridad, confidencialidad y disponibilidad de la información en el ecosistema **Layla**, se ha implementado un modelo de **Control de Acceso Basado en Roles (RBAC)**. Este esquema permite restringir las operaciones en el sistema distribuido basándose en la identidad y el contexto del usuario respecto a cada proyecto narrativo.

Justificación Técnica: El sistema utiliza un mecanismo de autenticación mediante **JSON Web Tokens (JWT)** gestionado por el Backend de .NET 10 y validado por **ASP.NET Core Identity**. La autorización no es estática; el sistema evalúa dinámicamente el rol del usuario (Escritor, Editor o Lector) según el ID del proyecto solicitado. Esto resuelve el reto de la **seguridad en sistemas distribuidos**, asegurando que las peticiones a los servicios de MongoDB (manuscritos) y Neo4j (grafos) solo se ejecuten si el token contiene los privilegios adecuados.

Módulo / Recurso	Acción	Escritor	Editor	Lector (Reg.)	Invitado	Admin
Cuentas y Auth	Registro e Inicio de Sesión	✓	✓	✓	✗	✓
Manuscritos (MongoDB)	Crear o eliminar proyectos	✓	✗	✗	✗	✓

	Modificar texto enriquecido	✓	✓	✗	✗	✓
Wiki y Tramas (Neo4j)	Gestionar nodos y relaciones	✓	✓	✗	✗	✓
	Visualizar mapas narrativos	✓	✓	✓	✗	✓
Módulo de Voz (WS)	Transmisión de audio	✓	✓	✗	✗	✓
	Recepción de audio	✓	✓	✓	✗	✓
Consultas Generales	Lectura de obras públicas	✓	✓	✓	✓	✓

Estrategia de Clientes y Especialización de Interfaces

Para abordar el reto de la **Heterogeneidad** en sistemas distribuidos, el proyecto Layla no busca replicar idénticas funcionalidades en todas las plataformas, sino que implementa una estrategia de **Especialización de Clientes**.

El sistema utiliza un API Gateway unificado que sirve a tres interfaces distintas y enruta las solicitudes hacia los servicios especializados según el dominio funcional:

Cliente de Escritorio (WPF / .NET 9): "La Herramienta de Producción"

- **Objetivo:** Proveer un entorno de "Deep Work" (trabajo profundo) para el escritor.
- **Justificación Técnica:** WPF permite acceso directo al sistema de archivos para caché local y un manejo superior de eventos de teclado y renderizado de texto en comparación con el DOM de un navegador.

Cliente Web (Blazor Server): "La Herramienta de Gestión"

- **Objetivo:** Administración, análisis de datos y acceso público.
- **Justificación Técnica:** Permite el acceso universal para Lectores y Administradores sin necesidad de instalaciones, ideal para dashboards y catálogos de lectura.

Cliente Movil (Android / Kotlin): "La Herramienta de Administración Ligera"

- **Objetivo:** Administración móvil de proyectos propios y consulta de métricas del sistema.
- **Justificación técnica:** Aprovecha Kotlin y Compose para una superficie móvil simple, rápida y mantenible, sin replicar el editor de manuscritos ni el módulo de voz.

Esta separación garantiza que el usuario siempre tenga la herramienta más adecuada para la tarea (movilidad, administración o productividad), manteniendo la consistencia de los datos mediante los servicios backend y sus almacenes especializados.

Distribución de Responsabilidades por Cliente

Característica	Escritorio (WPF)	Web (Blazor)	Movil (Android)
Autoria de manuscritos	Completa	Completa	No implementada
Voz	Oyente / Hablante	Oyente / Hablante	No implementada
Wiki y grafo	Creación, edición y visualización	Creación, edición y visualización	No implementado
Administración	No dedicado	Dashboard y usuarios	Métricas y proyectos propios

Aclaraciones importantes

Sobre contexto de usuarios: Un usuario puede ser **Escritor** de su propia obra y simultáneamente ser **Lector** en una obra ajena.

Manejo de invitados: Un **Invitado** tiene un alcance limitado a la lectura de contenido marcado como “público”.

Abordaje de Retos de Sistemas Distribuidos

El sistema Layla implementa estrategias específicas para mitigar los desafíos inherentes a la computación distribuida:

- **Heterogeneidad:** Se logra la interoperabilidad total entre los clientes (Android y Web) y los servicios de backend (ASP.NET Core y Node.js) mediante el intercambio de mensajes en formato JSON estándar y el uso de APIs RESTful agnósticas a la plataforma.

- **Concurrencia:** El sistema maneja múltiples escritores y editores trabajando simultáneamente. El backend de ASP.NET Core utiliza programación asíncrona para gestionar los hilos de WebSockets del chat de voz sin bloquear el servidor, mientras que Node.js maneja la concurrencia de I/O para la edición de wikis.

Resolución de Conflictos en Edición: Dado que la implementación de Transformación Operacional (OT) o Tipos de Datos Replicados Libres de Conflictos (CRDTs) excede el alcance temporal de este proyecto académico, Layla adopta un enfoque pragmático de **Last Write Wins (LWW)** para la persistencia de manuscritos.

- **Justificación:** Se asume que, en un entorno de escritura colaborativa controlada, los conflictos de edición en el mismo párrafo son poco frecuentes. LWW simplifica drásticamente la arquitectura de sincronización manteniendo la integridad del documento al priorizar la solicitud más reciente procesada por el servidor.
- **Manejo de Fallas:** Se han implementado mecanismos de resiliencia en los clientes, incluyendo colas de reintento automático para la reconexión de sockets y caché local temporal para prevenir la pérdida de datos de manuscritos ante intermitencias de red.

Uso de Arquitectura Orientada a Eventos

Comunicación Inter-Servicios: Arquitectura Orientada a Eventos Siguiendo las recomendaciones de diseño para sistemas distribuidos robustos, se utilizará un modelo de **Mensajería Asíncrona** utilizando **RabbitMQ**.

Flujo de Trabajo:

1. **Publicación (.NET):** Cuando ocurre una acción crítica (ej. "Crear Personaje"), el Backend Core publica un mensaje en la cola `worldbuilding_events`.
2. **Cola (RabbitMQ):** Actúa como buffer intermedio, garantizando la persistencia del mensaje incluso si el servicio consumidor no está disponible temporalmente.
3. **Consumo (Node.js):** El servicio de Worldbuilding escucha la cola, procesa el mensaje y actualiza Neo4j y MongoDB según corresponda.

Justificación: Este enfoque mejora la latencia percibida por el usuario (no hay espera de I/O bloqueante) y aumenta la resiliencia del sistema ante caídas parciales de servicios.

Análisis y Gestión de Riesgos Técnicos

El desarrollo de un sistema distribuido heterogéneo como **Layla** conlleva desafíos inherentes de consistencia, disponibilidad y complejidad. A continuación, se detallan los riesgos identificados y las estrategias de arquitectura diseñadas para mitigarlos.

Matriz de Riesgos y Mitigación

ID	Riesgo Identificado	Impacto	Probabilidad	Estrategia de Mitigación (Arquitectura)
R-01	Inconsistencia de Datos (Data Drift). Un fallo en el servicio de Node.js podría provocar que un personaje borrado en SQL Server siga existiendo en Neo4j (Grafo).	Alto	Media	Consistencia Eventual & Soft Deletes. Se utiliza RabbitMQ para garantizar que el mensaje de borrado llegue eventualmente. Además, se implementa "Borrado Lógico" (IsDeleted=true) en lugar de físico, facilitando la recuperación y sincronización.
R-02	Latencia en Voz (Jitter). La red puede saturarse, causando que el audio llegue entrecortado o con mucho retraso, afectando la experiencia de colaboración.	Medio	Alta	Modelo Push-to-Talk. Se elimina el eco y se reduce el ancho de banda requerido al permitir solo un canal de audio activo a la vez. El protocolo WebSockets sobre TCP garantiza el orden de llegada de los paquetes.

R-03	Agotamiento de Recursos en Desarrollo. Ejecutar 6 contenedores (SQL, Mongo, Neo4j, RabbitMQ, Backends) puede saturar la memoria RAM del entorno local.	Alto	Alta	Límites de Contenedores. Configuración estricta de mem_limit en Docker Compose (ej. SQL Server limitado a 1GB) y uso de imágenes base Alpine o Slim para reducir la huella de memoria.
R-04	Fallo de Orquestación (SPOF). Si el contenedor del Backend Core (.NET) falla, todo el sistema de autenticación y acceso se detiene.	Crítico	Baja	Políticas de Auto-Recuperación. Implementación de restart: unless-stopped en Docker para el reinicio automático del servicio y Health Checks que previenen el enrutamiento de tráfico a contenedores no saludables.
R-05	Dependencia de Plataforma (WPF). El cliente de escritorio queda atado exclusivamente al sistema operativo Windows.	Bajo	N/A (Aceptado)	Diseño de API Abierta. Se mitiga mediante una API RESTful estandarizada que permite, en el futuro, el desarrollo de clientes para otras plataformas (Mac/Linux) sin modificar el Backend.

R-06	Sobrescritura de Datos (Race Condition). Si dos editores modifican el mismo párrafo simultáneamente, los cambios de uno podrían perderse al aplicar Last Write Wins.	Medio	Baja	Bloqueo Optimista y LWW. El sistema favorece la simplicidad del modelo LWW. Para mitigar pérdidas críticas, el cliente WPF guarda un historial de versiones local (Local History) que permite al usuario recuperar texto sobrescrito manualmente si detecta una inconsistencia.
-------------	---	-------	------	--

Detalle de Estrategias Críticas

A. Estrategia ante la Inconsistencia (R-01) Dado que no es viable implementar transacciones distribuidas (Two-Phase Commit) por su impacto en el rendimiento, el sistema adopta un modelo de **Consistencia Eventual**.

- **Acción:** Cuando el Escritor realiza un cambio crítico (ej. Renombrar Proyecto), el Backend .NET actualiza su base de datos y publica un evento en **RabbitMQ**.
- **Garantía:** Si el servicio de grafos (Node.js) está caído, el mensaje persiste en la cola. Al recuperarse el servicio, procesará los mensajes pendientes en orden, sincronizando el estado del sistema automáticamente.

B. Estrategia de Seguridad Distribuida Para evitar la latencia de red en la validación de sesiones entre microservicios:

- **Token Autosuficiente:** Se utilizan **JWT (JSON Web Tokens)** firmados con una clave secreta compartida (HS256).
- **Inyección de Secretos:** La clave de firma se inyecta en ambos contenedores (.NET y Node.js) mediante variables de entorno seguras en tiempo de ejecución. Esto permite que el microservicio de Node.js valide la autenticidad de una petición sin necesidad de realizar una llamada síncrona al servicio de identidad de .NET.

Modelo de datos (Diagrama de entidad-relación)

Estrategia de Persistencia Políglota

Para cumplir con los requisitos de optimización de datos, el sistema utiliza tres motores de base de datos:

1. SQL Server: Para la integridad referencial de usuarios, roles y permisos.
2. MongoDB: Para el almacenamiento de documentos extensos (manuscritos) y estructuras variables (fichas de wiki).
3. Neo4j: Para modelar y consultar eficientemente las relaciones complejas entre personajes y líneas temporales del universo narrativo.

Descripciones de casos de uso

CU-01: Explorar Catálogo Público

Actor principal: Visitante / Lector

Precondición: El portal web está en línea y existen proyectos marcados como públicos.

Flujo básico:

1. El usuario accede a la página de inicio del portal.
2. El sistema presenta un catálogo de proyectos públicos con portada, título, género y resumen.
3. El usuario puede filtrar proyectos por género literario o buscar por título.
4. El usuario selecciona un proyecto de interés para conocer más detalles.

Flujos alternativos: FA-1: No hay proyectos públicos → el sistema muestra un mensaje informativo.

Postcondición: El usuario visualiza la lista de obras públicas disponibles.

CU-02: Previsualizar Sinopsis

Actor principal: Visitante / Lector

Precondición: El usuario está navegando el catálogo público (CU-01).

Flujo básico:

1. El usuario selecciona un proyecto del catálogo.
2. El sistema muestra la portada, el título, el autor, el género y la sinopsis de la obra.
3. El usuario puede regresar al catálogo o iniciar sesión para leer el contenido completo.

Flujos alternativos: FA-1: El proyecto fue despublicado mientras el usuario lo visualizaba → el sistema informa que la obra ya no está disponible.

Postcondición: El usuario conoce los detalles públicos de la obra seleccionada.

CU-03: Iniciar Sesión / Registrarse

Actor principal: Visitante

Precondición: El sistema está en línea. El usuario no ha iniciado sesión.

Flujo básico:

1. El usuario selecciona la opción de inicio de sesión o registro.
2. El sistema presenta el formulario correspondiente.
3. El usuario proporciona su correo electrónico y contraseña.
4. El sistema valida las credenciales e inicia la sesión del usuario.
5. El sistema redirige al usuario a su área de trabajo personal.

Flujos alternativos: FA-1: Credenciales incorrectas → el sistema notifica el error sin revelar cuál campo es incorrecto. FA-2: Correo no registrado → el sistema ofrece la opción de registrarse. FA-3: Registro con correo ya existente → el sistema informa que la cuenta ya existe.

Postcondición: El usuario está autenticado y accede a las funciones restringidas del sistema.

CU-04: Gestionar Perfil

Actor principal: Escritor / Editor / Lector Registrado

Precondición: El usuario está autenticado en el sistema.

Flujo básico:

1. El usuario accede a la sección de configuración de su cuenta.
2. El sistema muestra los datos actuales del perfil (nombre, correo, avatar).
3. El usuario modifica los campos deseados.
4. El sistema valida y guarda los cambios.
5. El sistema confirma la actualización del perfil.

Flujos alternativos: FA-1: Datos inválidos (nombre vacío) → el sistema señala el campo con error.

Postcondición: El perfil del usuario queda actualizado en el sistema.

CU-05: Crear Proyecto Narrativo

Actor principal: Escritor

Precondición: El usuario está autenticado y tiene rol de Escritor.

Flujo básico:

1. El usuario selecciona la opción de crear un nuevo proyecto.
2. El sistema muestra un formulario con campos: título, género, sinopsis y visibilidad.
3. El usuario completa el formulario y confirma la creación.
4. El sistema registra el proyecto y asigna al usuario como propietario.
5. El sistema notifica a los servicios relacionados para preparar el espacio de trabajo del proyecto.
6. El sistema redirige al usuario al área de trabajo del nuevo proyecto.

Flujos alternativos: FA-1: Título duplicado o vacío → el sistema solicita corrección antes de continuar.

Postcondición: El proyecto queda creado y el usuario es su propietario único.

CU-06: Gestionar Colaboradores

Actor principal: Escritor (propietario del proyecto)

Precondición: El usuario es propietario del proyecto. El proyecto existe.

Flujo básico:

1. El usuario accede a la configuración de colaboradores del proyecto.
2. El sistema muestra la lista de colaboradores actuales con sus roles.
3. El usuario invita a un colaborador ingresando su correo electrónico y asignándole un rol.
4. El sistema notifica al colaborador y agrega su acceso al proyecto.
5. El usuario puede modificar el rol de un colaborador existente o eliminarlo del proyecto.

Flujos alternativos: FA-1: El correo no corresponde a ningún usuario registrado → el sistema informa que no se encontró la cuenta. FA-2: Intento de asignar rol de Propietario → el sistema no permite esa asignación.

Postcondición: La lista de colaboradores y sus roles queda actualizada según las instrucciones del propietario.

CU-07: Configurar Privacidad del Proyecto

Actor principal: Escritor (propietario del proyecto)

Precondición: El usuario es propietario del proyecto.

Flujo básico:

1. El usuario accede a la configuración del proyecto.
2. El sistema muestra el estado de visibilidad actual (público / privado).

3. El usuario cambia la visibilidad del proyecto.
4. El sistema aplica el cambio y actualiza la disponibilidad del proyecto en el catálogo.

Flujos alternativos: FA-1: El proyecto contiene capítulos sin guardar → el sistema advierte antes de publicar.

Postcondición: La visibilidad del proyecto queda actualizada; si es público, aparece en el catálogo.

CU-08: Editar Manuscrito

Actor principal: Escritor / Editor

Precondición: El usuario está autenticado y tiene rol de Escritor o Editor en el proyecto.

Flujo básico:

1. El usuario selecciona un capítulo dentro del área de trabajo del proyecto.
2. El sistema carga el contenido más reciente del capítulo.
3. El usuario redacta o modifica el texto con formato enriquecido.
4. El sistema guarda automáticamente los cambios de manera periódica.
5. Si hay otros usuarios editando simultáneamente, el sistema muestra la posición del cursor de cada colaborador en tiempo real.
6. El sistema sincroniza los cambios entre todos los participantes activos.

Flujos alternativos: FA-1: Conflicto de edición (versión desactualizada) → el sistema aplica la política de última escritura gana e informa al usuario. FA-2: Pérdida de conexión → el sistema reintenta la sincronización al restablecerse.

Postcondición: El capítulo queda guardado con el contenido más reciente y todos los participantes ven el mismo estado.

CU-09: Gestionar Nodos de Wiki

Actor principal: Escritor / Editor

Precondición: El usuario está autenticado con rol de Escritor o Editor en el proyecto.

Flujo básico:

1. El usuario accede a la sección de Wiki del proyecto.
2. El sistema muestra el listado de entidades existentes (personajes, lugares, objetos, etc.).
3. El usuario crea, edita o elimina una entidad proporcionando nombre, tipo y descripción.

4. El sistema guarda los cambios y actualiza el grafo narrativo para reflejar la entidad.

Flujos alternativos: FA-1: Intento de eliminar una entidad referenciada en el grafo → el sistema solicita confirmación y advierte sobre las relaciones existentes.

Postcondición: La base de conocimiento del mundo narrativo queda actualizada con la entidad modificada.

CU-10: Visualizar Grafo Narrativo

Actor principal: Escritor / Editor / Lector Registrado

Precondición: El usuario está autenticado y tiene acceso al proyecto.

Flujo básico:

1. El usuario accede a la sección de Grafo Narrativo del proyecto.
2. El sistema presenta una visualización de nodos (entidades de la wiki) y aristas (relaciones entre ellas).
3. El usuario puede navegar por el grafo, seleccionar nodos y explorar sus conexiones.
4. El Escritor o Editor puede agregar nuevas relaciones entre entidades existentes.

Flujos alternativos: FA-1: El proyecto no tiene entidades en la wiki → el sistema muestra el grafo vacío con instrucciones para agregar entidades.

Postcondición: El usuario ha explorado las relaciones narrativas entre entidades del universo creativo.

CU-11: Iniciar Sesión de Voz (Hablaante)

Actor principal: Escritor / Editor

Precondición: El usuario está autenticado, tiene acceso al proyecto y su dispositivo tiene micrófono disponible.

Flujo básico:

1. El usuario accede a la sala de voz del proyecto.
2. El sistema muestra la lista de participantes presentes.
3. El usuario activa el modo de transmisión (mantiene pulsado el botón de hablar).
4. El sistema captura el audio del micrófono y lo transmite a los demás participantes en tiempo real.
5. Al soltar el botón, la transmisión se detiene.

Flujos alternativos: FA-1: Micrófono no disponible → el sistema informa que no puede acceder al audio y permite escuchar sin hablar.

Postcondición: El audio del usuario fue transmitido a los demás participantes conectados.

CU-12: Unirse a Sesión de Voz (Oyente)

Actor principal: Escritor / Editor / Lector Registrado

Precondición: El usuario está autenticado y tiene acceso al proyecto. Hay una sesión de voz activa.

Flujo básico:

1. El usuario accede a la sala de voz del proyecto.
2. El sistema conecta al usuario como oyente y muestra la lista de participantes.
3. El usuario escucha en tiempo real el audio de los hablantes activos.
4. El usuario puede abandonar la sala en cualquier momento.

Flujos alternativos: FA-1: No hay participantes en la sala → el usuario puede esperar o salir.

Postcondición: El usuario estuvo conectado como oyente durante la sesión de voz.

CU-13: Leer Historia Publicada

Actor principal: Lector Registrado / Visitante

Precondición: El proyecto está marcado como público y tiene capítulos disponibles.

Flujo básico:

1. El usuario selecciona una obra del catálogo.
2. El sistema presenta el listado de manuscritos y capítulos de la obra.
3. El usuario selecciona un capítulo para leer.
4. El sistema presenta el contenido del capítulo en modo lectura.
5. El usuario puede navegar entre capítulos.

Flujos alternativos: FA-1: El proyecto fue marcado como privado mientras el usuario leía → el sistema cierra el acceso y redirige al catálogo.

Postcondición: El usuario leyó el capítulo o capítulos seleccionados de la obra.

CU-14: Consultar Reportes del Sistema

Actor principal: Administrador

Precondición: El usuario está autenticado con rol de Administrador.

Flujo básico:

1. El administrador accede al panel de reportes.

2. El sistema muestra estadísticas de uso: usuarios registrados, proyectos creados, actividad reciente.
3. El administrador puede filtrar los datos por período de tiempo.
4. El sistema actualiza las métricas según los filtros aplicados.

Flujos alternativos: FA-1: No hay datos en el período seleccionado → el sistema muestra las métricas en cero.

Postcondición: El administrador visualizó el estado y actividad general de la plataforma.

CU-15: Gestionar Usuarios

Actor principal: Administrador

Precondición: El usuario está autenticado con rol de Administrador.

Flujo básico:

1. El administrador accede al panel de gestión de usuarios.
2. El sistema muestra el listado de cuentas registradas con estado y fecha de creación.
3. El administrador puede buscar un usuario por nombre o correo.
4. El administrador puede desactivar o eliminar una cuenta.
5. El sistema aplica los cambios y actualiza el estado del usuario afectado.

Flujos alternativos: FA-1: Intento de eliminar la cuenta del administrador en sesión → el sistema no lo permite. FA-2: El usuario a eliminar tiene proyectos activos → el sistema advierte antes de proceder.

Postcondición: La cuenta del usuario gestionado queda en el estado indicado por el administrador.

Construcción

Selección justificada de la pila tecnológica

Componente	Tecnología	Justificación y Rol en el Proyecto
Backend A (Core)	ASP.NET Core 10 (C#)	Gestiona autenticación, usuarios, proyectos, roles, SignalR y publicación de eventos mediante outbox.
Backend B (Worldbuilding)	Node.js 22 + Express 5 + TypeScript	Gestiona manuscritos, wiki y grafo narrativo usando MongoDB y Neo4j.

Comunicación entre servicios	RabbitMQ	Broker AMQP para eventos asíncronos, especialmente project.created y eventos de colaboración.
API Gateway	YARP .NET 9	Punto de entrada único para clientes; enruta rutas de core y worldbuilding.
Cliente Web	Blazor Server .NET 9	Cliente de autoría completo y superficie principal de administración.
Cliente Movil	Android Kotlin + Compose	Cliente reducido para login, administración de proyectos propios y consulta de métricas.
Cliente Escritorio	WPF .NET 9	Cliente rico para trabajo intensivo de escritura, wiki, grafo, voz y colaboración.
Base de Datos 1	SQL Server	Persistencia relacional para usuarios, proyectos, roles y outbox.
Base de Datos 2	MongoDB	Persistencia documental para manuscritos, capítulos, versiones y wiki.
Base de Datos 3	Neo4j	Modelo de grafo para relaciones entre entidades narrativas.
Infraestructura	Docker, Vagrant y Puppet	Contenerización y despliegue académico en entorno reproducible de múltiples servicios.

Aplicación de cliente rico

El cliente de escritorio está desarrollado con WPF (Windows Presentation Foundation) sobre .NET 9. Esta elección prioriza la productividad del escritor sobre la portabilidad: WPF provee renderizado de texto acelerado por hardware, acceso directo al sistema de archivos para caché local y control nativo del micrófono, sin capa intermediaria de navegador.

La arquitectura sigue el patrón MVVM (Model-View-ViewModel) mediante CommunityToolkit.Mvvm, con inyección de dependencias provista por Microsoft.Extensions.DependencyInjection. El tema visual es un sistema de diseño neumórfico personalizado (NeumorphismTheme.xaml) con tipografía Inter y paleta de colores coherente con la identidad de Layla.

Dependencias NuGet

Paquete	Versión	Propósito
CommunityToolkit.Mvvm	8.2.2	Patrón MVVM: RelayCommand, ObservableObject, mensajería entre ViewModels
Microsoft.AspNetCore.SignalR.Client	9.0.3	Comunicación en tiempo real: edición colaborativa, presencia y sala de voz
NAudio	2.2.1	Captura y reproducción de audio para el módulo Push-to-Talk (sala de voz)
SkiaSharp	3.119.2	Renderizado 2D del grafo narrativo (nodos y aristas sobre Canvas)
Microsoft.Extensions.DependencyInjection	10.0.8	Contenedor de inyección de dependencias para servicios y repositorios
FluentWPF	0.10.2	Componentes de UI adicionales con soporte de estilos Fluent Design
Microsoft.Extensions.Logging	10.0.8	Registro de eventos y errores en entorno de desarrollo

Aplicación de servicios (API) - Servidor

El backend de Layla está compuesto por dos microservicios heterogéneos, cada uno con su propia pila tecnológica, base de datos y dominio de datos.

server-core — ASP.NET Core 10.0

Servicio principal de autenticación, gestión de proyectos y comunicación en tiempo real. Construido sobre ASP.NET Core 10 con arquitectura en capas (Api / Core / Infrastructure). Publica eventos al message broker mediante el patrón Outbox-after-commit.

Paquete	Versión	Propósito
Microsoft.AspNetCore.Authentication.JwtBearer	10.0.3	Validación de tokens JWT en cada petición HTTP autenticada

Microsoft.AspNetCore.Identity.EntityFrameworkCore	10.0.3	Gestión de usuarios, contraseñas con hashing y bloqueo de cuentas
Microsoft.EntityFrameworkCore.SqlServer	10.0.3	ORM para SQL Server: entidades AppUser, Project, ProjectRole
RabbitMQ.Client	6.8.1	Publicación de eventos al exchange worldbuilding.events (patrón Outbox)
MongoDB.Driver	3.6.0	Lectura cruzada de datos de worldbuilding desde server-core
MailKit	4.16.0	Envío de correos de verificación de cuenta vía SMTP
Swashbuckle.AspNetCore	6.5.0	Generación automática de documentación Swagger/OpenAPI
System.IdentityModel.Tokens.Jwt	8.16.0	Generación y firma criptográfica de tokens JWT (HMAC-SHA512)

server-worldbuilding — Node.js 22 + Express 5 + TypeScript 5.9.3

Servicio de dominio narrativo: manuscritos, wiki de entidades y grafo de relaciones. Construido sobre Express 5 con TypeScript estricto (strict: true). Consume eventos del message broker para mantener consistencia eventual con server-core. Incluye análisis estático con ESLint 10 y cobertura de pruebas con Vitest.

Paquete / Herramienta	Versión	Propósito
express	5.2.1	Framework web HTTP; manejo de rutas, middleware y respuestas
mongoose	9.3.3	ODM para MongoDB: modelos Manuscript, WikiEntry, ChapterVersion

neo4j-driver	6.0.1	Cliente Neo4j para consulta y escritura del grafo narrativo
amqplib	0.10.9	Consumidor RabbitMQ: eventos project.created y collaborator.*
jsonwebtoken	9.0.3	Validación del JWT recibido del api-gateway en cada solicitud
zod	4.3.6	Validación y parseo de esquemas de entrada en controladores
helmet + express-rate-limit	8.1.0 / 8.3.2	Headers de seguridad HTTP y limitación de tasa de peticiones
ESLint + typescript-eslint	10.1.0 / 8.57.2	Análisis estático de código TypeScript con reglas estrictas
Vitest + @vitest/coverage-v8	2.1.9	Framework de pruebas unitarias con cobertura de código

Estándares de codificación

C#

El desarrollo en C# se alineará estrictamente con las directrices oficiales establecidas por el creador del lenguaje.

- **Estándar de Referencia:** [C# Coding Conventions \(Microsoft\)](#) y las [Framework Design Guidelines](#).
- **Justificación:** La adopción de las convenciones de Microsoft asegura que el código sea idiomático, predecible para cualquier desarrollador del ecosistema .NET y compatible con las herramientas de análisis estático de código (como Roslyn) integradas en los entornos de desarrollo modernos.

JavaScript

Para el desarrollo en JavaScript, se adoptará uno de los estándares más estrictos y ampliamente utilizados en la industria para mitigar las ambigüedades inherentes al lenguaje.

- **Estándar de Referencia:** [Airbnb JavaScript Style Guide](#).
- **Justificación:** Esta guía proporciona un enfoque razonable y exhaustivo para escribir código JavaScript moderno (ES6 y superior). Impone reglas estrictas sobre la mutabilidad, el manejo de tipos y la sintaxis, lo que reduce

drásticamente la probabilidad de errores en tiempo de ejecución y facilita la configuración de *linters* (como ESLint).

Kotlin

El código escrito en Kotlin deberá seguir las pautas idiomáticas definidas por los creadores del lenguaje, priorizando la concisión y la seguridad contra nulos.

- **Estándar de Referencia:** [Kotlin Coding Conventions \(JetBrains\)](#) y, en su caso, la [Android Kotlin Style Guide \(Google\)](#).
- **Justificación:** Seguir las convenciones oficiales de JetBrains garantiza el uso correcto de las características funcionales y orientadas a objetos de Kotlin. Además, asegura la compatibilidad con el formateador de código oficial (ktlint) y maximiza la interoperabilidad fluida con bases de código existentes en Java.

Reportes de análisis estático de código

Prácticas de construcción realizadas

La siguiente tabla muestra el estado de implementación de cada módulo funcional por cliente. El cliente de escritorio (WPF) y el cliente web (Blazor Server) son los canales principales de autoría; el cliente Android actúa como plataforma reducida de administración de proyectos y métricas.

Leyenda: ✓ Implementado completamente | Parcial | – No aplica en esta plataforma

Modulo / Funcionalidad	Escritorio (WPF)	Web (Blazor)	Android
Autenticación: login y registro	Completo	Completo	Completo
Lista de proyectos propios	ProjectListView	MyProjects	MyProjectsScreen
Crear / editar / eliminar proyecto	Completo	Completo	Completo
Configuración de privacidad	Completo	Completo	Parcial, desde formulario de proyecto
Catalogo público de proyectos	PublicProjectsView	ExploreProjects	No implementado como pantalla dedicada
Gestión de colaboradores	WorkspaceView	CollaboratorsModal	No implementado

Editor de manuscritos	ManuscriptEditorView (RTF)	RichTextEditor	No implementado
Edición colaborativa en tiempo real	SignalR + debounce	SignalR	No implementado
Historial / restauración de versiones	ChapterDiffWindow	VersionHistoryModal	No implementado
Wiki de entidades narrativas	WikiEntityEditorView	WikiTab	No implementado
Grafo narrativo	NarrativeGraphView	GraphTab	No implementado
Sala de voz Push-to-Talk	VoicePanelView + SignalR	VoiceRoom / VoiceDrawer	No implementado
Lector de historia publicada	ReaderWorkspaceView	ProjectWorkspace modo lectura	No implementado
Panel de administración	No implementado	Admin/Dashboard	No implementado
Gestión de usuarios (Admin)	No implementado	Admin/ManageUser	No implementado
Reportes del sistema	No implementado	Dashboard + StatCard	Parcial, metricas si el usuario es admin
Configuración de perfil	SettingsView	Settings	No implementado

Pruebas

Plan de pruebas para la Aplicación de servicios (API) y clientes

Se ha diseñado un plan de pruebas para cubrir tanto la lógica de negocio como los requisitos no funcionales de concurrencia:

- **Pruebas Unitarias (.NET):** Se utiliza **xUnit** junto con **NSubstitute** para validar los servicios de autenticación y lógica de negocio en el Backend A (ASP.NET Core).
- **Pruebas de Integración (Node.js):** Se emplea **vitest** para verificar la correcta integración de los endpoints de la Wiki con MongoDB y Neo4j.

- **Pruebas de Carga y Concurrency:** Uso de **k6** para simular múltiples conexiones simultáneas al módulo de voz y verificar la estabilidad de los WebSockets bajo carga.
- **Cobertura Objetivo:** Se busca asegurar un mínimo del 70% de cobertura de código en los módulos críticos del sistema.

Procedimiento de prueba (automatizado)

Las suites de prueba están organizadas por servicio y se ejecutan de forma independiente. Las pruebas unitarias no requieren servicios externos activos, ya que todas las dependencias (bases de datos, mensajería) están sustituidas por dobles de prueba.

1. Pruebas unitarias — server-core y client-web (.NET / xUnit)

Requisito previo: .NET SDK 10.0 o superior instalado.

```
cd src/server-core
dotnet test --logger "console;verbosity=normal"
```

El comando ejecuta todos los proyectos de prueba de la solución. Los resultados se muestran en la consola con el nombre de cada prueba y su estado (passed / failed / skipped).

2. Pruebas unitarias — server-worldbuilding (Node.js / Vitest)

Requisito previo: Node.js 22 y pnpm instalados.

```
cd src/server-worldbuilding
pnpm install
pnpm test          # ejecución única
pnpm test:watch    # modo observación (desarrollo)
pnpm test:coverage # genera reporte de cobertura en /coverage
```

El reporte de cobertura se genera en formato HTML en src/server-worldbuilding/coverage/index.html.

3. Pruebas de carga — k6

Requisito previo: k6 instalado (<https://k6.io>). Los servicios deben estar en ejecución.

```
k6 run tests/load/auth.js      # carga de autenticación
k6 run tests/load/projects.js  # carga de gestión de proyectos
k6 run tests/load/manuscripts.js # carga de manuscritos
k6 run tests/load/signalr.js   # carga de conexiones SignalR
k6 run tests/load/scenarios.js # escenario realista combinado
```

El escenario combinado (scenarios.js) simula 50 usuarios virtuales concurrentes con distribución realista: 40% tráfico público, 30% autenticado, 20% edición y 10% SignalR.

Casos de prueba (automatizados)

La suite de pruebas automatizadas cubre los dos servicios backend y el cliente web. En total comprende 323 pruebas unitarias distribuidas en 19 archivos de prueba.

Resumen por componente

Componente	Framework	Archivos	Total pruebas
server-core (Layla.Core.Tests)	xUnit 2.9.2 + NSubstitute 5.3.0	10	187
client-web (client- web.Tests)	xUnit 2.9.2	2	5
server-worldbuilding (__tests__)	Vitest 2.1.9	7	131
TOTAL		19	323

Pruebas server-core (C# / xUnit)

AuthServiceTests (25 pruebas)

- LoginAsync: usuario no existe, cuenta bloqueada, contraseña incorrecta, credenciales válidas
- LoginAsync: comportamiento del TokenVersion (incremento y fallo)
- RegisterAsync: correo duplicado, contraseña rechazada, registro exitoso
- RegisterAsync: envío de correo de verificación con PIN generado

TokenServiceTests (16 pruebas)

- GenerateToken: claims sub, email, tokenVersion presentes y correctos
- GenerateToken: roles múltiples, sin roles, algoritmo HMAC-SHA512
- GenerateToken: expiración dentro de ventana configurada, JTI único por token
- Constructor: secreto nulo lanza InvalidOperationException

ProjectServiceTests (42 pruebas)

- CreateProjectAsync: éxito con rol OWNER, fallo de base de datos
- UpdateProjectAsync: no propietario (Forbidden), proyecto inexistente, actualización exitosa
- DeleteProjectAsync: no propietario, eliminación exitosa
- GetUserProjectsAsync: retorna proyectos con roles correctos por usuario

- InviteCollaboratorAsync: no propietario, email no encontrado, ya miembro, rol inválido (OWNER/unknown), editor invitado
- JoinPublicProjectAsync: proyecto privado, proyecto inexistente
- RemoveCollaboratorAsync: intento de eliminar al propietario

PresenceTrackerTests (27 pruebas)

- MarkActive: primera conexión de propietario, proyecto ya activo, solo lector, múltiples conexiones
- MarkActive: actualización y no degradación de rol, preservación de avatar
- MarkInactive: conexión desconocida, último participante, otros presentes, dos conexiones mismo usuario
- IsProjectActive: proyecto sin registro, solo lector, editor presente
- GetActiveParticipants: proyecto no encontrado, participante con userId/displayName/role/avatarUrl
- GetUserConnection: usuario desconocido, usuario conectado, después de desconexión

VoiceRoomManagerTests (29 pruebas)

- AddParticipant: DTO con userId/displayName/isSpeaking/role correcto, crea sala, reemplaza estado
- RemoveParticipant: proyecto no encontrado, usuario no en sala, éxito
- RemoveByConnectionId: encontrado (projectId/userId/removes), no encontrado
- SetSpeaking: proyecto no encontrado, usuario no en sala, activa, desactiva
- GetParticipants: proyecto no encontrado, dos participantes
- TryConsumeAudioSlot: primera llamada, reintento inmediato bloqueado

TokenVersionValidatorTests (8 pruebas)

- ContextFails: principal nulo, claim userId faltante, claim tokenVersion faltante, usuario no encontrado, versión desactualizada
- ContextDoesNotFail: token válido
- Agrega NameIdentifier cuando está ausente, no lo duplica

OutboxProcessorWorkerTests (13 pruebas)

- Sin mensajes: publish y saveChanges nunca llamados
- ClientEvicted: publish llamado, routing key correcto, projectId/userId en payload, marcado como procesado
- EventType desconocido: publish no llamado, marcado como procesado

- Publisher lanza excepción: mensaje no procesado, mensajes restantes siguen procesándose
- Múltiples mensajes: SaveChanges llamado exactamente una vez

AppUserServiceTests (17 pruebas)

- GetAllAppUsersAsync: usuarios existentes (1 DTO por usuario), fallo de repositorio
- GetAppUserByIdAsync: encontrado (email/displayName mapeados), no encontrado
- UpdateAppUserAsync: éxito con displayName actualizado, no encontrado
- DeleteAppUserAsync / BanAppUserAsync: éxito

GlobalExceptionMiddlewareTests (6 pruebas)

- Next exitoso: código HTTP sin cambio
- Next lanza excepción: HTTP 500, Content-Type JSON, body con statusCode/errorMessage/traceId

RequireUserIdFilterTests (4 pruebas)

- Endpoint anónimo: result no asignado
- Claim userId faltante: result 401 Unauthorized
- Claim userId presente: result no asignado, userId en HttpContext.Items

Pruebas client-web (C# / xUnit)

AuthStateProviderTests (2 pruebas):

- GetAuthenticationStateAsync_ReturnsAnonymous_WhenTokenIsMissing
- GetAuthenticationStateAsync_ProjectsNameAndRoleClaims

ApiClientTests (3 pruebas):

- SendAsync_ClearsSessionAndThrows_OnUnauthorized
- SendAsync_DeserializesCamelCaseJson
- SendAsync_ThrowsApiException_OnInvalidJson

Pruebas server-worldbuilding (TypeScript / Vitest)

Auth.test.ts (27 pruebas)

- authenticate middleware: sin header (401), esquema Basic (401), token expirado (401), firma inválida (401), token válido (next + req.user)
- optionalAuth middleware: sin header (next sin user), token válido (user seteado), token inválido (next igual)
- requireRole middleware: sin req.user (401), rol no permitido (403), rol correcto (next)

ManageJWT.test.ts (8 pruebas)

- verifyAccessJWTToken: sub → id, preserva email y role, id explícito tiene prioridad sobre sub
- Sin id ni sub: id vacío, token expirado lanza TokenExpiredError, secreto diferente lanza JsonWebTokenError

ProjectGuard.test.ts (14 pruebas)

- requireWriteAccess: READER (403), undefined (403), EDITOR (next), OWNER (next)
- requireProjectAccess: sin projectId (next), sin req.user (401)
- Con Neo4j: OWNER (next + role), EDITOR (next + role), sin registro Neo4j y sin token (403)

Graph.service.test.ts (18 pruebas)

- getGraph: llama repo.getGraph con projectId, filtra por entityType, retorna nodos
- createRelationship: ambas entidades existen (true + campos), entidad faltante (false)
- deleteRelationship: delegación a repo, propagación de error
- getEntityAppearances: vacío, un registro con chapterTitle y manuscriptId

Manuscript.service.test.ts (19 pruebas)

- createManuscript: orden por conteo existente, orden explícito, título correcto, sin campo content
- getFullStoryByProject: manuscritos en orden, capítulos en orden, incluye content
- updateChapter: no existe (sin conflicto), timestamp viejo (conflicto + estado actual)
- updateChapter: timestamp nuevo (llama repo), sin timestamp (sin verificación de conflicto)
- createChapter: orden por conteo, orden explícito, chapterId no vacío, content y title correctos

WikiEntry.service.test.ts (17 pruebas)

- createEntry: éxito con sync Neo4j (mergeEntity llamado, marcada synced), tolerante a fallo Neo4j
- updateEntry: no existe (null), éxito con re-sync Neo4j, tolerante a fallo Neo4j
- deleteEntry: no encontrado MongoDB (false, deleteEntity no llamado)
- deleteEntry: éxito primer intento, fallo todos los reintentos (retorna true, 3 intentos), éxito segundo intento (2 llamadas)

Mention.service.test.ts (28 pruebas)

- stripRtf: cadena vacía, texto sin RTF, control words, escape hex \e9 → é, unicode \u233, espacios dobles colapsados
- extractMentions: lista vacía, sin coincidencia, coincidencia exacta (entityId/name/type), case-insensitive
- extractMentions: sin match parcial de palabra, una mención por entidad, entry name vacío/muy largo ignorado
- syncChapterMentions: wiki vacía, contenido coincide (un mention + syncAppearances llamado)
- syncChapterMentions: sin coincidencias, entidad eliminada filtrada, syncAppearances lanza error

Resultados

A continuación, se presentan los resultados de la ejecución de las suites de prueba unitaria en el entorno de desarrollo local. Las pruebas de carga (k6) requieren servicios en ejecución y sus resultados se reportan por separado como evidencia de despliegue.

server-core y client-web (.NET / xUnit)

Suite	Pasadas	Fallidas	Omitidas	Tiempo
Layla.Core.Tests + client-web.Tests	187	0	0	7.2402 Segundos

Resultado: PASS – todas las pruebas correctas

[xUnit.net 00:00:04.46] Finished: Layla.Core.Tests

La serie de pruebas se ejecutó correctamente.

Pruebas totales: 187

Correcto: 187

Tiempo total: 7.2402 Segundos

server-worldbuilding (Node.js / Vitest)

Suite	Archivos	Pasadas	Fallidas	Duración
server-worldbuilding (__tests__)	7	131	3	3.67s

Resultado: FAIL

Nota: las salidas stderr en WikiEntry.service.test.ts son mensajes de log intencionales generados por los casos de prueba de resiliencia (Neo4j caído / reintentos). No indican errores; todas las pruebas pasan.

✓ src/__tests__/Mention.service.test.ts (28 tests) 62ms
 ✓ src/__tests__/WikiEntry.service.test.ts (17 tests) 128ms
 ✓ src/__tests__/Manuscript.service.test.ts (19 tests) 104ms
 ✓ src/__tests__/ManageJWT.test.ts (8 tests) 49ms
 ✓ src/__tests__/Auth.test.ts (27 tests) 39ms
 Test Files 7 passed (7)
 Tests 131 passed (131)
 Duration 3.67s (transform 2.21s, setup 0ms, collect 4.68s, tests 456ms, environment 5ms, prepare 6.27s)

Pruebas de carga — k6

Las pruebas de carga se ejecutan contra el sistema desplegado. Los umbrales definidos son:

Script	VUs / Etapas	Umbral de aceptación	Métrica
auth.js	20 VUs 30 s → 50 VUs 1 min → 0	p95 < 500 ms	POST /api/tokens/login
projects.js	Rampa hasta 50 VUs	p95 < 500 ms, p99 < 1 s	GET / POST /api/projects
manuscripts.js	Rampa hasta 50 VUs	p95 < 800 ms	Lectura y escritura de capítulos
signalr.js	Rampa hasta 50 VUs	p95 < 1 s	Conexiones WebSocket SignalR
scenarios.js	50 VUs combinados	p95 < 1 s global	Escenario mixto realista

Estrategia de despliegue

Arquitectura de Contenedores

El despliegue del ecosistema Layla se basa en una arquitectura de microservicios containerizados utilizando **Docker**. Se ha seleccionado una imagen base de **Debian Slim** para los servicios de backend (.NET y Node.js) debido a su equilibrio entre ligereza y compatibilidad con librerías de audio (ALSA) necesarias para el módulo de voz.

Orquestación y Ciclo de Vida (Docker Compose)

La orquestación de los servicios se define mediante un archivo docker-compose.yml que gestiona la interdependencia entre los contenedores. Se han configurado políticas de reinicio automático (restart: unless-stopped) para garantizar que, ante un fallo crítico o reinicio del servidor host, los servicios de backend y bases de datos se recuperen automáticamente sin intervención manual (Self-Healing).

Mecanismos de Resiliencia y Alta Disponibilidad

Para cumplir con los requisitos no funcionales de estabilidad (RNF-08, RNF-09, RNF-10), se implementan las siguientes estrategias:

- **Verificaciones de Salud (Health Checks):** Se configuran sondas de salud en los contenedores de base de datos (SQL Server, MongoDB). Los servicios dependientes (Backends) esperan mediante la directiva depends_on: condition: service_healthy hasta que la persistencia esté totalmente operativa, evitando errores de conexión durante el arranque (Cold Start).
- **Resiliencia Transitoria (Polly en .NET):** Se utiliza la librería Polly y la resiliencia nativa de Entity Framework Core (EnableRetryOnFailure) para manejar micro-cortes de red. El sistema reintenta automáticamente las operaciones fallidas contra la base de datos antes de reportar un error al usuario.
- **Persistencia de Datos (Volúmenes):** Se utilizan volúmenes nombrados de Docker para desacoplar los datos de los contenedores de ejecución, garantizando que la información de los manuscritos y usuarios sobreviva a la destrucción o actualización de los contenedores de base de datos.

Conclusiones

El sistema desarrollado se distingue por la integración de un ecosistema tecnológico amplio y moderno, en el que convergen plataformas como .NET Core en su versión 10 para la construcción de servicio backend, RabbitMQ como mecanismo de mensajería asíncrona, y herramientas de despliegue y automatización como Docker y Jenkins. A ello se suma la implementación de interfaces web mediante Blazor Pages y .NET Core en su versión 9, enriquecidas con TypeScript y Tailwind, así como clientes específicos desarrollados en Kotlin y WPF. La arquitectura se complementa con la utilización de motores de datos heterogéneos como: SQL Server, MongoDB y Neo4J. Estos permiten responder de manera óptima a distintos tipos de almacenamiento y consulta. Esta diversidad tecnológica asegura flexibilidad, escalabilidad y capacidad de adaptación frente a escenarios de negocio variados.

La documentación arquitectónica, sustentada en diagramas de despliegue, arquitectónicos, de secuencia, de actividad y de máquina de estados, ha permitido validar la coherencia del diseño y garantizar que los componentes interactúen de manera clara y predecible. Dicho proceso contribuye a reducir riesgos de integración y facilita el mantenimiento futuro del sistema. Asimismo, la incorporación de requisitos no funcionales, tales como la latencia, el tiempo máximo de recuperación y la cobertura de pruebas, refuerza la resiliencia y la escalabilidad de la solución, asegurando que el crecimiento no comprometa la calidad.

La interoperabilidad entre bases de datos relacionales, documentales y de grafos constituye un elemento diferenciador, pues permite optimizar el rendimiento según la naturaleza de la información y el caso de uso. En paralelo, la experiencia de usuario se ve fortalecida por la disponibilidad de interfaces multicanal, accesibles desde entornos web y de escritorio, lo que garantiza consistencia y modernidad en la interacción.

Un aspecto innovador del proyecto es la incorporación de herramientas de inteligencia artificial, como Claude, que han sido utilizadas para potenciar la productividad en el desarrollo, la documentación y el soporte. Esta integración abre la posibilidad de futuras funcionalidades inteligentes que incrementen el valor del sistema.

Finalmente, la definición de métricas técnicas, de negocio y de calidad asegura que la solución no solo cumpla con los requisitos funcionales, sino que también se alinee con objetivos estratégicos de sostenibilidad, eficiencia operativa y satisfacción del usuario.

Apéndices

Apéndice A: Roadmap Tecnológico y Evolución a Producto Comercial (Layla 2.0)

Este documento anexo describe la visión a largo plazo para la plataforma **Layla**, proyectando su evolución desde la actual arquitectura académica distribuida hacia un producto comercial unificado y escalable.

Justificación de la Arquitectura Actual vs. Futura

La arquitectura actual (Layla v1.0) fue diseñada para satisfacer los requisitos de **heterogeneidad, sistemas distribuidos y redes**, priorizando la implementación de múltiples pilas tecnológicas (.NET, Node.js, Kotlin, WPF) para demostrar competencias de integración compleja.

Sin embargo, para una etapa de comercialización y mantenimiento a largo plazo (Layla v2.0), la estrategia técnica cambiará hacia la **Unificación del Código (Code Sharing)** para reducir la deuda técnica y acelerar el despliegue de nuevas funcionalidades.

Propuesta de Stack Tecnológico Unificado (Layla v2.0)

El objetivo de la siguiente iteración será migrar de un ecosistema fragmentado a un **Monorepo basado en TypeScript**, manteniendo la robustez del Backend actual, pero unificando las tres interfaces de usuario (Web, Escritorio, Móvil).

Evolución Tecnológica

Componente	Layla v1.0 (Académico/Actual)	Layla v2.0 (Visión Comercial)	Motivación del Cambio
Cliente Web	Blazor Server (.NET)	SvelteKit + TypeScript	Migrar de renderizado en servidor (SSR) a una SPA reactiva moderna que elimine la recarga de páginas y mejore la experiencia de escritura.
Cliente Escritorio	WPF (Windows)	Tauri + Svelte (TypeScript)	Reemplazar la dependencia exclusiva de Windows por una arquitectura multiplataforma (Win/Mac/Linux) ultraligera, compartiendo el 90% del código con la web.
Cliente Móvil	Android Nativo (Kotlin)	Capacitor / Ionic	Permitir el despliegue simultáneo en iOS y Android utilizando la misma base de código web, reduciendo el equipo de desarrollo necesario.
Protocolo de Voz	WebSockets (TCP Raw)	WebRTC	Adoptar el estándar industrial para comunicación en tiempo

			real (P2P), delegando la gestión de códecs, cancelación de eco y latencia al navegador/WebView.
Backend	.NET 10 + Node.js	.NET 10 (Unificado)	Mantener .NET como núcleo por su rendimiento, evaluando la migración de los servicios de Node.js a .NET 10 (Minimal APIs) para simplificar la infraestructura de despliegue.

Estrategia de Migración

La transición hacia Layla 2.0 se ejecutará en tres fases post-académicas:

1. **Fase de Unificación de UI (Frontend):** Desarrollo de una biblioteca de componentes de interfaz en **Svelte** (Botones, Editor de Texto, Paneles) que reemplace las componentes de Blazor y los controles XAML de WPF.
2. **Fase de Hibridación (Desktop):** Implementación de **Tauri** para encapsular la nueva aplicación web de Svelte, permitiendo el acceso al sistema de archivos local (Local First) necesario para el trabajo offline.
3. **Fase de Estandarización de Audio:** Sustitución del módulo de WebSockets manual por una implementación de **WebRTC**, mejorando la calidad de audio y reduciendo la carga en el servidor central (Server-Load) al pasar a una topología Peer-to-Peer donde sea posible.

Conclusión del Apéndice

Mientras que Layla v1.0 demuestra la capacidad de orquestar sistemas dispares y resolver retos de bajo nivel (sockets, hilos, serialización manual), Layla v2.0 se enfocará en la **Eficiencia del Desarrollador (Developer Experience)** y la **Escalabilidad del Producto**, alineándose con las tendencias modernas de desarrollo de software SaaS (Software as a Service).